

МИНОБРНАУКИ РОССИИ
Федеральное государственное бюджетное образовательное
учреждение высшего образования
«Ярославский государственный университет имени П. Г. Демидова»

Кафедра дискретного анализа

Сдано на кафедру

«_____» _____ 2026 г.

Заведующий кафедрой,

д. ф.-м. н., доцент

_____ А. В. Николаев

Выпускная квалификационная работа

**Разработка игрового приложения
«Коридор» с модулем стратегического поведения
соперника**

по направлению

01.04.02 Прикладная математика и информатика

Научный руководитель

д. ф.-м. н., доцент

_____ А. В. Николаев

«_____» _____ 2026 г.

Студент группы ИВТ-21МО

_____ А. О. Карпунин

«_____» _____ 2026 г.

Ярославль, 2026

Реферат

Объем 92 с., 4 гл., 0 рис., 0 табл., 10 источников, 6 прил.

Ключевые слова: **игры, Коридор, граф, поиск кратчайшего пути, игровой искусственный интеллект, алгоритм минимакса, альфа-бета отсечение, метод Монте-Карло, функция оценки, обучение весов.**

Объект исследования — логическая игра «Коридор» и алгоритмы автоматического расчета лучшего хода в ней.

Цель работы — разработка игрового приложения, с помощью которого пользователь сможет сыграть против одного из четырёх программных соперников, а также сравнить эти алгоритмы между собой.

Результат работы — приложение, реализующее правила игры «Коридор», с возможностью игры против компьютера или наблюдения за игрой алгоритмов друг против друга.

Содержание

Введение	5
1. Игра «Коридор»	7
1.1. Необходимые определения	7
1.2. Описание игры	7
1.3. Модуль стратегического поведения	8
2. Алгоритмы	10
2.1. Общие функции и структура алгоритмов	10
2.2. Случайный алгоритм	14
2.3. Алгоритм минимакса	15
2.4. Минимакс с альфа-бета отсечением	15
2.5. Метод Монте-Карло (MCTS)	18
2.6. Сводные параметры сложности	19
3. Игровое приложение	21
3.1. Техническая реализация	21
3.2. Взаимодействие с приложением	22
4. Сравнение алгоритмов	26
4.1. Методика сравнения	26
4.2. Собираемые метрики	26
4.3. План эксперимента	27
4.4. Форма представления результатов	27
4.5. Анализ результатов	28
4.6. Ограничения эксперимента	29
Заключение	30
Список литературы	31
Приложение А. Реализация интерфейса алгоритма	32
Приложение Б. Реализация случайного алгоритма	47
Приложение В. Реализация алгоритма MiniMax	51
Приложение Г. Реализация алгоритма AlphaBeta	59

Приложение Д. Реализация алгоритма Monte Carlo	73
Приложение Е. Реализация обучения весов функции оценки	85

Введение

Коридор — настольная логическая игра для двух или четырёх игроков, выпущенная в 1997 году компанией Gigamic. Она была создана Мирко Маркези на основе более ранней игры «Блокада». Игра основана на последовательном перемещении фишек по клеточному полю 9×9 и установке перегородок, изменяющих доступные маршруты к целевой стороне доски [4]. Несмотря на лаконичность правил, пространство состояний игры «Коридор» чрезвычайно велико: по различным оценкам, число возможных позиций превышает 10^{42} , что ставит задачу разработки сильного программного соперника в один ряд с задачами для классических игр вроде шахмат [5].

Общий вид партии показан на рисунке 1. На каждом ходу игрок выбирает одно из двух действий: продвинуть свою фишку к противоположной стороне поля или поставить перегородку, усложняющую маршрут соперника. Количество перегородок ограничено, а правила запрещают полностью блокировать путь любого игрока к его целевой стороне поля.

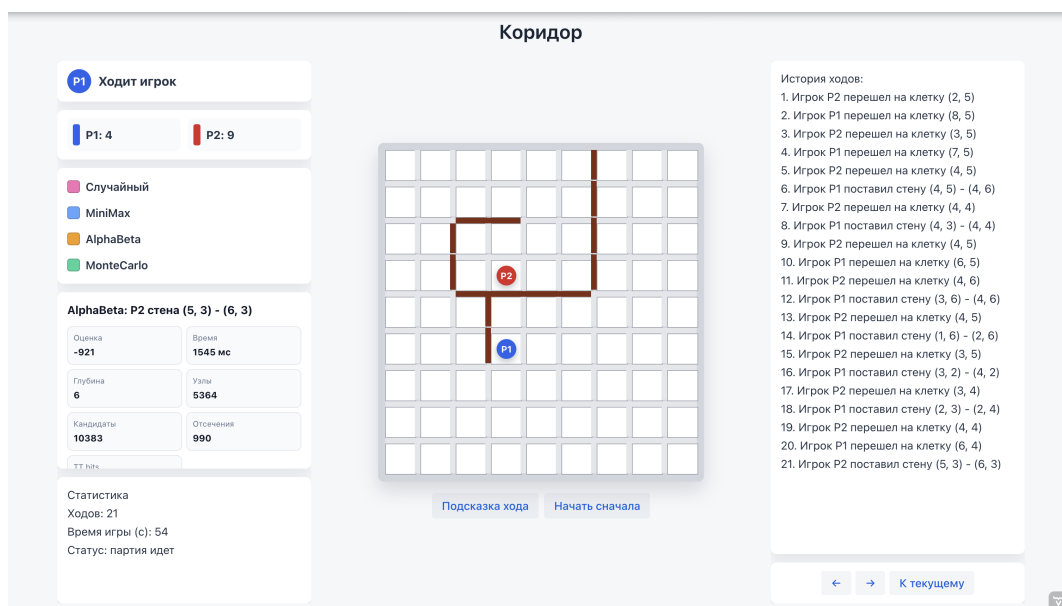


Рис. 1 — Пример партии в приложении «Коридор»

Игра получила заметное распространение среди настольных логических игр; в частности, она входила в подборку Games 100 журнала *Games* [3]. Для данной работы важен не только прикладной интерфейс игры, но и математическая структура задачи: «Коридор» удобно рассматривать как задачу принятия решений на графе. Клетки поля соответствуют вершинам графа $G = (V, E)$, допустимые переходы между соседними клетками — рёбрам. Установленная перегородка удаляет из E одно или несколько рёбер. Ключевое ограничение правил — перегородка не

может полностью перекрыть путь ни одного из игроков к его целевой линии — требует после каждого потенциального изменения структуры графа проверять наличие хотя бы одного маршрута у обоих участников.

Задача разработки программного соперника для подобных игр является классической в области игрового искусственного интеллекта. Для игр с полной информацией и нулевой суммой, к которым относится «Коридор», традиционно применяются алгоритмы на основе дерева игровых состояний: минимакс и его расширения [7]. Альтернативным подходом является метод Монте-Карло (MCTS) [8], не требующий явной оценочной функции.

Целью выпускной квалификационной работы является разработка игрового приложения «Коридор», включающего реализацию правил игры, визуализацию игрового поля и модуль стратегического поведения, объединяющий четыре алгоритма программного соперника. Проведённое сравнение алгоритмов в серии автоматических партий позволяет оценить их относительную силу.

Для реализации цели работы были поставлены следующие задачи:

1. изучить правила игры «Коридор» и построить графовую модель игрового поля,
2. реализовать алгоритм проверки корректности перемещения фишки, включая прыжки через соперника,
3. реализовать алгоритм проверки корректности установки перегородки с проверкой достижимости целевых линий,
4. реализовать алгоритм поиска кратчайшего пути до целевой линии на основе обхода графа в ширину,
5. разработать многофакторную функцию оценки игровой позиции с настраиваемыми весами и механизмом их онлайн-коррекции по результатам партий,
6. реализовать четыре алгоритма программного соперника: случайный выбор, минимакс, минимакс с альфа-бета отсечением и MCTS,
7. провести сравнение алгоритмов в серии автоматических партий,
8. разработать интерфейс игрового приложения с поддержкой режимов «Игрок против ИИ», «ИИ против ИИ», режима подсказок и воспроизведения партии.

1. Игра «Коридор»

1.1. Необходимые определения

В формулировке правил игры и последующем описании алгоритмов встречается несколько понятий, которые нуждаются в пояснении.

Игровым полем называется квадратная доска размером $n \times n$ клеток, по которым перемещаются фишки игроков. В стандартном варианте $n = 9$; приложение также поддерживает поля 7×7 и 11×11 . Клетки нумеруются парами (i, j) , где $0 \leq i, j \leq n - 1$.

Фишкой называется объект, расположенный ровно в одной клетке поля. В двухпользовательском варианте игры участвуют две фишки.

Перегородкой называется элемент, размещаемый на границе между клетками. Перегородка занимает два смежных межклеточных промежутка и может быть ориентирована горизонтально (блокирует переходы вверх–вниз между двумя парами клеток) или вертикально (блокирует переходы влево–вправо). Каждый игрок располагает фиксированным числом перегородок, зависящим от размера поля: 8 для поля 7×7 , 10 для 9×9 , 12 для 11×11 .

Целевой линией игрока называется строка, противоположная от его стартовой позиции. Первый игрок стартует из центра последней строки и стремится к строке $i = 0$; второй — из центра первой строки и стремится к строке $i = n - 1$.

Состоянием игры называется кортеж $(p_1, p_2, W, w_1, w_2, t)$, где p_1, p_2 — позиции фишек; W — множество установленных перегородок; w_1, w_2 — количества оставшихся перегородок у каждого игрока; $t \in \{1, 2\}$ — номер игрока, совершающего текущий ход.

Графовой моделью игрового поля является неориентированный граф $G = (V, E)$, где V — множество клеток ($|V| = n^2$), а E — множество допустимых переходов между соседними клетками. Установка перегородки удаляет из E два ребра. В приложении граф хранится в виде хеш-таблицы: каждой клетке (i, j) сопоставлен объект `BoardTile` с четырьмя булевыми полями — флагами доступности переходов в четырёх направлениях: влево, вперёд (убывание i), вправо и назад (возрастание i).

1.2. Описание игры

Игра «Коридор» была выпущена компанией Gigamic в 1997 году. Впоследствии она была переведена на несколько языков и распространилась по всему миру, став предметом изучения в области игрового искусственного интеллекта.

В начале партии фишки расставляются на противоположных сторонах поля: первый игрок — в центре нижней строки, второй — в центре верхней строки. Игроки ходят по очереди, начиная с того, кому выпал жребий. Цель каждого игрока — первым достичь любой клетки своей целевой линии.

За один ход игрок выполняет ровно одно из двух действий.

Перемещение фишки. Фишка перемещается на соседнюю клетку по горизонтали или вертикали при условии, что между ними нет перегородки. Если соседняя клетка занята фишкой соперника и за ней нет перегородки и нет границы поля, разрешается прыжок через соперника (фишка перемещается через него). Если прямой прыжок невозможен из-за перегородки или края поля, допускается диагональный ход: фишка перемещается на клетку, расположенную под углом 90° относительно направления прыжка, при условии отсутствия перегородки в этом направлении.

Установка перегородки. Игрок выбирает позицию и ориентацию перегородки. Установка допускается, если: у игрока есть хотя бы одна неиспользованная перегородка; новая перегородка не пересекается и не совпадает с уже установленными; после её установки у каждого игрока сохраняется хотя бы один маршрут к своей целевой линии.

Партия завершается, когда фишка одного из игроков достигает любой клетки целевой линии. Этот игрок объявляется победителем.

1.3. Модуль стратегического поведения

Модуль стратегического поведения представляет собой программный компонент, отвечающий за выбор хода программным соперником. Он получает на вход текущее состояние игры, количество оставшихся перегородок у каждого из игроков и возвращает выбранный ход.

Требования к модулю формулируются следующим образом.

1. *Корректность:* выбираемый ход всегда должен быть допустимым с точки зрения правил игры. В приложении реализован дополнительный уровень защиты: если алгоритм по какой-либо причине возвращает недопустимый ход, GameProcessor автоматически подменяет его наилучшим доступным ходом фишкой, выбранным по эвристике близости к целевой линии и штрафа за повторение позиций.
2. *Завершённость:* алгоритм должен завершаться за конечное время. Все переборные алгоритмы (минимакс, альфа-бета, MCTS) работают с ограничением по времени: каждый из них получает лимит в миллисекундах и прекращает поиск по его истечении.
3. *Единый интерфейс:* все стратегии реализуют интерфейс Algorithm с методом `getMove(board, hardnessLevel, playerId, wallsLeft1, wallsLeft2)`

Это позволяет переключать алгоритмы без изменения остальных компонентов приложения.

4. *Настраиваемая сложность*: каждый алгоритм поддерживает три уровня сложности — EASY, MEDIUM и HARD, которые влияют на глубину поиска, лимит времени и число итераций.
5. *Масштабируемость*: архитектура допускает добавление новых алгоритмов без изменения существующих компонентов через `ComputerPlayerFabric`.

В рамках данной работы в модуле реализованы четыре стратегии. Случайный алгоритм (`RandomAlgorithm`) равновероятно выбирает ход из множества допустимых и служит базовой линией для сравнения. Минимакс (`MinimaxAlgorithm`) перебирает дерево игровых состояний на ограниченную глубину без каких-либо отсечений. Альфа-бета (`AlphaBetaAlgorithm`) реализует минимакс с отсечениями, сортировкой ходов и таблицей транспозиций. Метод Монте-Карло (`MonteCarloAlgorithm`) оценивает позиции через многократные случайные симуляции с использованием формулы УСТ.

2. Алгоритмы

2.1. Общие функции и структура алгоритмов

Все четыре алгоритма программного соперника реализуют интерфейс `Algorithm`, содержащий как абстрактные методы, так и конкретные реализации вспомогательных функций по умолчанию (`default`-методы Java). Такой подход исключает дублирование кода и гарантирует единообразное поведение алгоритмов в одинаковых ситуациях.

Генерация допустимых ходов

Метод `getMoves(board, playerId, wallsLeft)` формирует полное множество допустимых ходов из текущего состояния. Оно складывается из двух частей.

Первая часть — допустимые перемещения фишки. Они получаются вызовом `board.getAvailableMoves(pos)`, который обходит четыре направления от текущей позиции, проверяя флаги `BoardTile`, и обрабатывает прыжки через соперника: прямой прыжок добавляется, если за фишкой соперника нет перегородки и нет края поля; в противном случае добавляются диагональные прыжки в обе стороны при их доступности.

Вторая часть — допустимые установки перегородок. Если у игрока не осталось перегородок (`wallsLeft <= 0`), эта часть пуста. В противном случае алгоритм формирует множество *кандидатных клеток*, ограничивая полный перебор стратегически значимой областью поля. Кандидатами являются: первые 12 клеток кратчайшего пути соперника к его целевой линии; первые 7 клеток кратчайшего пути самого алгоритма; окрестность радиуса 2 вокруг фишки соперника; окрестность радиуса 1 вокруг первых 5 клеток пути соперника. Для каждой кандидатной клетки проверяются два возможных направления перегородки — вертикальное и горизонтальное. Перегородка включается в список кандидатов, если она физически размещается (`isWallPlaceable`) и не отсекает ни одного из игроков от его целевой линии (`isValidWall`), причём выполняется хотя бы одно из условий стратегической ценности: перегородка удлиняет путь соперника не менее чем на 2 хода; соперник находится в 3 ходах от победы и перегородка его замедляет; перегородка удлиняет путь соперника не меньше, чем собственный путь алгоритма. Финальный список перегородок сортируется по убыванию функции `wallImpact`.

Поиск кратчайшего пути

Метод `calculateShortestPath(board, start, targetRow)` реализует обход графа в ширину (BFS) от стартовой позиции до ближайшей клетки целевой строки. Алгоритм использует очередь уровней: на каждой итерации внешнего цикла обрабатывается весь текущий уровень, после чего счётчик глубины увеличивается на 1. Возвращается длина кратчайшего пути в ходах; если путь не найден — возвращается значение 1000 (условная бесконечность). Трудоемкость одного вызова составляет $O(|V| + |E|)$, что при фиксированном размере поля эквивалентно $O(1)$ по параметрам задачи.

Метод `getShortestPathCells` дополняет BFS восстановлением пути через хранение родителей и используется при генерации кандидатных клеток для установки перегородок.

Эндшпильные правила

Метод `findEndgameMove` применяется всеми четырьмя алгоритмами как первый шаг перед запуском основного поиска. Если из текущей позиции фишку можно переместить прямо на целевую линию — этот ход немедленно возвращается без какого-либо перебора. Если такого хода нет, но соперник находится не далее чем в двух ходах от победы и у алгоритма есть перегородки — ищется перегородка с максимальным значением `wallImpact`, замедляющая соперника.

Предпочтение направления движения

Метод `movementPreference` добавляет к оценке хода бонус или штраф в зависимости от направления и истории позиций:

- ход вперёд (в сторону целевой линии): +45 очков;
- ход назад: −120 очков;
- приближение к целевой линии на Δ клеток: $+35 \cdot \Delta$ очков;
- повторение позиции, встречавшейся 1–2 хода назад: −500 очков;
- повторение более давней позиции: −180 очков.

История позиций хранится для каждого игрока отдельно в `GameProcessor` и передаётся в алгоритм через метод `setRecentPositions`.

Функция оценки позиции

Функция оценки `evaluate(board, playerId, size, wallsLeft1, wallsLeft2)` является центральным компонентом всех переборных алгоритмов. Она ставит в соответствие каждому нетерминальному состоянию игры числовое значение, характеризующее его выгодность для программного соперника. Если состояние терминальное — победа алгоритма или пользователя — функция возвращает `+WIN_SCORE+`

d или $-\text{WIN_SCORE} - d$ соответственно, где $\text{WIN_SCORE} = 100\,000$, а d — текущая глубина поиска (это обеспечивает предпочтение более близкой победы).

Для нетерминальных состояний оценка вычисляется как взвешенная сумма семи признаков:

$$F(s) = \sum_{k=1}^7 \lambda_k \cdot f_k(s),$$

где λ_k — веса, а $f_k(s)$ — значения признаков. Признаки и их смысл перечислены в таблице 2.1.

Таблица 2.1

Признаки оценочной функции

k	Признак $f_k(s)$	Описание
1	$d_p - d_a$	Разность длин кратчайших путей: путь пользователя минус путь алгоритма
2	mob_a	Количество допустимых ходов фишкой алгоритма
3	$-\text{mob}_p$	Отрицательное количество допустимых ходов пользователя
4	$w_a - w_p$	Преимущество алгоритма по числу оставшихся перегородок
5	$\text{prog}_a - \text{prog}_p$	Разность продвижений к целевым линиям
6	$e(d_a)$	Эндшпильный бонус за близость алгоритма к победе
7	$-e(d_p)$	Эндшпильный штраф за близость пользователя к победе

Продвижение prog_a вычисляется как количество строк, пройденных фишкой алгоритма от стартовой позиции до текущей. Эндшпильная функция $e(d)$ принимает значения:

$$e(d) = \begin{cases} 10, & d \leq 1, \\ 4, & d = 2, \\ 1, & d = 3, \\ 0, & d > 3. \end{cases}$$

Веса λ_k по умолчанию заданы следующим образом (класс `EvaluationWeights`):

$$\lambda_1 = 65, \quad \lambda_2 = 4, \quad \lambda_3 = 3, \quad \lambda_4 = 18, \quad \lambda_5 = 6, \quad \lambda_6 = 165, \quad \lambda_7 = 175.$$

Онлайн-коррекция весов

Приложение реализует механизм автоматической коррекции весов функции оценки по результатам сыгранных партий (класс `EvaluationWeightsStore`). После завершения каждой партии, в которой участвовал программный соперник, вычисляется градиентная оценка полезности ходов.

В течение партии для каждого хода алгоритма формируется объект `EvaluationLearn` хранящий вектор признаков до хода и после него (`EvaluationFeatures before`, `after`). После завершения партии для каждого образца вычисляется дельта признаков $\Delta f_k = f_k^{\text{after}} - f_k^{\text{before}}$ и суммируется в градиент с учётом знака:

$$g_k = \sum_{\text{ходы алгоритма}} \text{reward} \cdot \Delta f_k, \quad \text{reward} = \begin{cases} +1, & \text{алгоритм победил,} \\ -1, & \text{алгоритм проиграл.} \end{cases}$$

Веса обновляются по правилу:

$$\lambda_k \leftarrow \text{clamp}(\lambda_k + \text{sign}(g_k) \cdot \Delta\lambda, \lambda_k^{\min}, \lambda_k^{\max}),$$

где $\Delta\lambda = 1$ — шаг обучения, $[\lambda_k^{\min}, \lambda_k^{\max}]$ — допустимый диапазон каждого веса, предотвращающий вырождение оценки. Актуальные веса сохраняются в файл `data/evaluation-weights.properties` и загружаются при следующем запуске приложения, что обеспечивает накопление опыта между сессиями. На момент написания работы механизм принял 15 обновлений весов на 1388 обучающих образцах, а текущие сохранённые веса составляют:

$$\lambda_1 = 120, \quad \lambda_2 = 7, \quad \lambda_3 = 12, \quad \lambda_4 = 8, \quad \lambda_5 = 25, \quad \lambda_6 = 186, \quad \lambda_7 = 170.$$

Сравнение начальных и сохранённых весов показывает, что механизм увеличил приоритет разности путей ($\lambda_1 : 65 \rightarrow 120$), штрафа за мобильность соперника ($\lambda_3 : 3 \rightarrow 12$), продвижения к цели ($\lambda_5 : 6 \rightarrow 25$) и собственно-го эндшпильного преимущества ($\lambda_6 : 165 \rightarrow 186$), одновременно снизив вес преимущества по оставшимся перегородкам ($\lambda_4 : 18 \rightarrow 8$).

Функция `wallImpact`

Функция `wallImpact(board, move, playerId, size)` оценивает стратегическую ценность конкретной установки перегородки. Она применяет ход к копии доски и вычисляет изменения длин путей:

$$\text{wallImpact} = (d_p^{\text{after}} - d_p^{\text{before}}) \cdot 100 - \max(0, d_a^{\text{after}} - d_a^{\text{before}}) \cdot 45,$$

то есть каждый шаг, добавленный к пути соперника, оценивается в 100 единиц, а каждый шаг, добавленный к собственному пути, штрафует 45 единицами. Перегородка считается стратегически ценной при $wallImpact > 0$.

2.2. Случайный алгоритм

Случайный алгоритм (RandomAlgorithm) является простейшим программным соперником и используется как базовая линия для сравнения. При каждом вызове метода `getMove` алгоритм первым делом применяет эндшпильное правило: если можно немедленно выиграть или срочно заблокировать соперника, он делает это независимо от уровня сложности. Если эндшпильное правило не сработало, поведение определяется уровнем сложности.

На уровне EASY алгоритм равновероятно выбирает ход из полного множества $M(s)$ допустимых ходов.

На уровне MEDIUM ходы сортируются по убыванию значения `evaluateAfterMove` (оценка позиции после совершения хода), и случайно выбирается один из первой половины списка.

На уровне HARD случайно выбирается один из трёх лучших ходов по той же оценке.

Алгоритм 1: Случайный алгоритм

```

1  эндшпильный ход  $\leftarrow$  findEndgameMove( $s$ );
2  if эндшпильный ход найден then
3    |   return эндшпильный ход
4  end
5   $M \leftarrow$  getMoves( $s$ );
6  if уровень = EASY then
7    |    $m^* \leftarrow$  случайный элемент  $M$ ;
8  else if уровень = MEDIUM then
9    |   сортировать  $M$  по убыванию evaluateAfterMove;
10   |    $m^* \leftarrow$  случайный элемент из первой половины  $M$ ;
11 else
12   |   сортировать  $M$  по убыванию evaluateAfterMove;
13   |    $m^* \leftarrow$  случайный элемент из первых трёх элементов  $M$ ;
14 end
15 return  $m^*$ 

```

Трудоёмкость алгоритма на уровне EASY определяется только генерацией ходов. Оценка каждого хода требует одного вызова BFS, поэтому на уровнях MEDIUM и HARD трудоёмкость составляет $O(|M(s)| \cdot |V|)$.

2.3. Алгоритм минимакса

Алгоритм минимакса (MinimaxAlgorithm) основан на предположении об оптимальной игре обоих участников [7]. На ходах программного соперника выбирается ход с максимальной оценкой (максимизирующий узел); на ходах пользователя предполагается выбор хода с минимальной оценкой (минимизирующий узел).

Алгоритм использует итеративное углубление: последовательно запускает поиск с глубиной $d = 1, 2, \dots, d_{\max}$ до истечения лимита времени. Результат последнего завершённого поиска возвращается как ответ. Это обеспечивает всегда иметь допустимый ответ, даже если время закончилось прежде, чем поиск на очередной глубине был завершён.

Максимальная глубина $d_{\max}$ и лимит времени T зависят от уровня сложности и размера поля:

Таблица 2.2

Параметры минимакса

Уровень	$d_{\max}$ (поле 9×9)	T , мс (поле 9×9)
EASY	3	700
MEDIUM	4	1800
HARD	5	3600

В целях ограничения времени работы ширина перебора ограничена функцией `limitMoves`: если число допустимых ходов превышает 16, список обрезается до первых 16 (порядок определяется генератором ходов). Для мемоизации использует таблица `cache`: ключ включает строковое представление доски, глубину, флаг максимизации и количества перегородок.

Трудоёмкость в наихудшем случае — $O(b^d)$, где $b \leq 16$ — ограниченная ширина перебора. При $d = 5$ это не более $16^5 \approx 1,05 \cdot 10^6$ узлов, что укладывается в лимит 3600 мс.

2.4. Минимакс с альфа-бета отсечением

Алгоритм альфа-бета (AlphaBetaAlgorithm) является наиболее сложным из реализованных. Он расширяет минимакс тремя независимыми механизмами ускорения: отсечениями, сортировкой ходов и таблицей транспозиций.

Альфа-бета отсечение

Алгоритм поддерживает два параметра: α — наилучшая оценка, гарантированная максимизирующему игроку, и β — наилучшая оценка, гарантированная минимизирующему. Если в ходе перебора $\beta \leq \alpha$, оставшиеся ходы в данном

узле можно не рассматривать: они не изменят итогового выбора. Это позволяет в лучшем случае сократить число рассматриваемых состояний до $O(b^{d/2})$.

Сортировка ходов

Эффективность отсечений критически зависит от порядка рассмотрения ходов: если лучший ход рассматривается первым, отсечения происходят как можно раньше. Функция `scoreMove` оценивает каждый ход до его применения по нескольким критериям:

- +20 000, если ход совпадает с записью из таблицы транспозиций (*лучший ход* из предыдущего поиска);
- +10 000 / +8 000, если ход является первым или вторым *killer-ходом* данного уровня глубины;
- вес из таблицы `goodMoves` — истории ходов, вызывавших отсечения в предыдущих узлах;
- значение функции оценки после применения хода.

После сортировки список ходов обрезаается до 36 кандидатов.

Таблица транспозиций

Таблица транспозиций (`transpositionTable`) хранит результаты уже вычисленных позиций. Ключ — строковое представление доски, флаг текущего игрока и количества перегородок. Каждая запись содержит оценку, глубину поиска, тип границы (точная оценка EXACT, нижняя граница LOWER или верхняя граница UPPER) и лучший ход для данной позиции. При попадании в таблицу алгоритм:

- если глубина записи $\geq$ текущей и тип EXACT — немедленно возвращает сохранённое значение;
- если тип LOWER — использует запись для ужесточения нижней границы α ;
- если тип UPPER — для ужесточения верхней границы β .

Затухающий поиск (Quiescence Search)

В позициях, где один из игроков находится не далее 2 ходов от победы (*шумные позиции*, `isNoisyPosition`), применяется расширенный поиск `quiescence`. Вместо немедленного возврата статической оценки алгоритм дополнительно рассматривает финальные ходы (перемещение фишки на целевую линию) и до 8 наиболее ценных установок перегородок с глубиной расширения 2. Это предотвращает ошибки горизонта — ситуации, когда статическая оценка не отражает угрозы, возникающей сразу после горизонта поиска.

Параметры алгоритма в зависимости от уровня сложности для поля 9×9 :

Алгоритм 2: Альфа-бета с таблицей транспозиций

```
1 Функция alphabeta( $s, d, \alpha, \beta, isMax$ ):  
2   if запись ( $s, d$ )  $\in TT$  then  
3     скорректировать  $\alpha, \beta$  по типу записи;  
4     if  $\alpha \geq \beta$  then  
5       return оценку из  $TT$   
6     end  
7   end  
8   if terminal( $s$ ) then  
9     return терминальная оценка  
10  end  
11  if  $d = 0$  then  
12    if isNoisyPosition( $s$ ) then  
13      return quiescence( $s, \alpha, \beta, isMax, 2$ )  
14    else  
15      return  $F(s)$   
16    end  
17  end  
18   $M \leftarrow$  первые 36 ходов из getMoves( $s$ ), отсортированных по  
   scoreMove;  
19  best  $\leftarrow -\infty$  если isMax, иначе  $+\infty$ ;  
20  foreach  $m \in M$  do  
21     $v \leftarrow$  alphabeta(apply( $s, m$ ),  $d - 1, \alpha, \beta, \neg isMax$ );  
22    обновить best,  $\alpha$  или  $\beta$ ;  
23    if  $\beta \leq \alpha$  then  
24      обновить killer-ходы и goodMoves;  
25      break ; // отсечение  
26    end  
27  end  
28  сохранить (best,  $d$ , тип) в  $TT$ ;  
29  return best
```

Параметры альфа-бета алгоритма

Уровень	$d_{\max}$	T , мс
EASY	4	1300
MEDIUM	6	3600
HARD	8	7600

2.5. Метод Монте-Карло (MCTS)

Алгоритм Монте-Карло (MonteCarloAlgorithm) строит статистику по многократным симуляциям от текущей позиции до конца партии [8]. В отличие от минимакса и альфа-бета, он не перебирает дерево равномерно до фиксированной глубины, а постепенно концентрирует вычислительный бюджет на наиболее перспективных ветвях. В реализации дополнительно используется эвристическая оценка позиции: она входит в UCT как небольшой нормализованный член и применяется при выборе ходов во время rollout. Структура алгоритма — дерево узлов, каждый из которых соответствует некоторому состоянию игры и хранит счётчики посещений n_v и побед w_v .

Основной цикл алгоритма состоит из четырёх фаз.

Выбор (Selection). Начиная с корня, алгоритм спускается по дереву, на каждом шаге выбирая дочерний узел с максимальным значением UCT:

$$\text{UCT}(v) = \frac{w_v}{n_v} + C \cdot \sqrt{\frac{\ln n_{\text{parent}}}{n_v}} + h(v),$$

где $C = 1,2$ — константа исследования; $h(v)$ — нормализованная эвристическая оценка $0,15 \cdot \tanh(F(s_v)/500)$, добавляющая информацию о позиции без полного доминирования над статистикой. Если узел ни разу не посещался, ему присваивается приоритет $+\infty$.

Расширение (Expansion). Когда все допустимые ходы из узла уже рассматривались в дочерних узлах, выбранный узел помечается как полностью расширенный. Иначе из списка ещё не рассмотренных ходов (предварительно отсортированных по scoreMove) берётся первый, и создаётся новый дочерний узел.

Симуляция (Simulation). Из нового узла проводится случайная игра до терминального состояния или до исчерпания лимита шагов rolloutDepth. На каждом ходу симуляции применяется эвристика epsilon-greedy:

1. если есть ход, немедленно завершающий партию — он выбирается;
2. если соперник находится не далее 2 ходов от победы и у текущего игрока есть перегородки — выбирается перегородка с максимальным wallImpact (если её wallImpact > 0);
3. с вероятностью 0,22 — случайный ход из полного списка;

4. иначе — из случайной выборки 18 ходов выбирается один из четырёх лучших по `scoreRolloutMove`.

Обратное распространение (Backpropagation). Результат симуляции распространяется вверх: для каждого узла на пути от листа к корню инкрементируются n_v и w_v (если результат является победой корневого игрока).

После истечения лимита времени или достижения бюджета итераций выбирается ход с наибольшим значением $n_c + w_c/n_c$ среди дочерних узлов корня (*robustChild*), что отдаёт предпочтение хорошо исследованным ходам.

Параметры алгоритма для поля 9×9 :

Таблица 2.4

Параметры MCTS

Уровень	Бюджет итераций	T , мс	Глубина симуляции
EASY	520	950	$9 \cdot 4 = 36$
MEDIUM	1700	2900	$9 \cdot 7 = 63$
HARD	3400	6200	$9 \cdot 10 = 90$

Алгоритм 3: MCTS

```

1  $r \leftarrow$  корневой узел с состоянием  $s$ ;
2 while не истёк лимит  $T$  и итераций  $< N$  do
3    $v \leftarrow \text{select}(r)$ ; // спуск по УСТ
4   if  $\neg \text{terminal}(v.s)$  then
5      $v \leftarrow \text{expand}(v)$ ; // новый дочерний узел
6   end
7    $\text{result} \leftarrow \text{simulate}(v, \text{rolloutDepth})$ ;
8    $\text{backpropagate}(v, \text{result})$ ;
9   if  $r.n > 300$  и  $\max_c(w_c/n_c) > 0.97$  then
10    break
11  end
12 end
13 return  $\arg \max_{c \in \text{children}(r)} (n_c + w_c/n_c)$ 

```

Условие досрочного выхода ($\text{bestWinRate} > 0.97$ при более чем 300 посещениях корня) позволяет экономить время в позициях, где один из ходов явно доминирует по статистике.

2.6. Сводные параметры сложности

Для удобства настройки экспериментов все уровни сложности сведены в таблицу 2.5. Значения приведены для стандартного поля 9×9 , поскольку именно оно используется в базовом сравнении алгоритмов. Для полей 7×7 и 11×11 параметры масштабируются в коде: на малом поле AlphaBeta может увеличивать

глубину поиска, а на большом поле MiniMax и AlphaBeta уменьшают глубину либо увеличивают временной лимит, чтобы партия оставалась интерактивной.

Таблица 2.5

Параметры уровней сложности алгоритмов для поля 9×9

Алгоритм	Уровень	Параметры
Random	EASY	Случайный выбор среди всех допустимых ходов
Random	MEDIUM	Случайный выбор из верхней половины ходов по функции оценки
Random	HARD	Случайный выбор из трёх лучших ходов по функции оценки
MiniMax	EASY	Глубина 3, лимит 700 мс, ширина перебора до 16 ходов
MiniMax	MEDIUM	Глубина 4, лимит 1800 мс, ширина перебора до 16 ходов
MiniMax	HARD	Глубина 5, лимит 3600 мс, ширина перебора до 16 ходов
AlphaBeta	EASY	Глубина 4, лимит 1300 мс, сортировка ходов, до 36 кандидатов
AlphaBeta	MEDIUM	Глубина 6, лимит 3600 мс, таблица транспозиций, killer-ходы
AlphaBeta	HARD	Глубина 8, лимит 7600 мс, quiescence search в шумных позициях
MCTS	EASY	До 520 итераций, лимит 950 мс, глубина rollout $9 \cdot 4 = 36$
MCTS	MEDIUM	До 1700 итераций, лимит 2900 мс, глубина rollout $9 \cdot 7 = 63$
MCTS	HARD	До 3400 итераций, лимит 6200 мс, глубина rollout $9 \cdot 10 = 90$

3. Игровое приложение

3.1. Техническая реализация

Игровое приложение написано на языке программирования Java версии 17 с использованием фреймворка Spring Boot версии 3.2.5, а также библиотеки Vaadin версии 24.4.2 [10], обеспечивающей отрисовку игрового поля и взаимодействие с пользователем через веб-интерфейс. Для сокращения шаблонного кода используется библиотека Lombok. Система сборки — Maven.

Архитектура приложения разделена на несколько независимых слоёв. Общая схема взаимодействия слоёв представлена на рисунке 2.



Рис. 2 — Общая архитектура приложения

Модель (`ru.ac.uniyar.model`) содержит классы, описывающие игровые объекты. Класс `Position` хранит координаты клетки (i, j) и предоставляет метод `move(rowDelta, colDelta)` для вычисления соседней позиции. Класс `BoardTile` описывает одну клетку поля четырьмя булевыми флагами доступности переходов. Класс `Board` хранит граф поля в виде `HashMap<Position, BoardTile>`, позиции фишек обоих игроков и реализует все операции с доской: размещение

перегородок (`placeWall`), получение доступных ходов (`getAvailableMoves`), обход для BFS (`getPathNeighbors`) и полное копирование (`copy`). Класс `Move` описывает ход: его тип (`MOVE_PLAYER` или `PLACE_WALL`), идентификатор игрока и позиции начала и конца. Класс `Game` хранит конфигурацию текущей партии: размер поля (`GameSize`), ссылки на игроков, номер текущего хода, время начала и флаг завершения.

Игроки (`ru.ac.uniyar.model.players`) реализуют иерархию через абстрактный класс `Player` с методом `getMove`. Класс `HumanPlayer` возвращает `null` (ход запрашивается через UI); класс `ComputerPlayer` делегирует вызов конкретному объекту `Algorithm`. Фабрика `ComputerPlayerFabric` создаёт нужную реализацию алгоритма по типу и уровню сложности.

Алгоритмы (`ru.ac.uniyar.model.algorithms`) описаны в главе 2. Все реализуют интерфейс `Algorithm`. Класс `AlgorithmReport` является неизменяемой записью с результатами последнего хода: выбранный ход, оценка, достигнутая глубина, число рассмотренных узлов, число отсечений, число попаданий в таблицу транспозиций, время в миллисекундах и текстовое пояснение. Класс `EvaluationFeatures` хранит вектор признаков состояния; `EvaluationWeights` — веса; `EvaluationWeightsStore` управляет загрузкой и сохранением весов.

Сервисный слой (`ru.ac.uniyar.service`) включает три компонента. `BoardFactory` создаёт начальное состояние доски заданного размера. `GameRules` проверяет корректность установки перегородки через BFS и конвертирует координаты рендера в игровые координаты. `GameProcessor` — аннотированный `@SessionScope` Spring-компонент, управляющий всем жизненным циклом партии: запуск (`startNewGame`), выполнение ходов (`makeMove`, `makeComputerMove`), режим воспроизведения (`replayPrevious`), сбор образцов для обучения весов и получение подсказок от всех четырёх алгоритмов. `BoardAnalyzer` формирует текстовые пояснения к ходам ИИ. `TournamentService` реализует автоматические серии партий для турнира и сравнения алгоритмов.

Интерфейсный слой (`ru.ac.uniyar.ui`) реализован через компоненты Vaadin. `StartPageController` отображает стартовый экран с выбором режима и параметров. `GamePageController` реализует основной экран игры: отрисовку доски в виде сетки Div-элементов, панель информации (`GameInfoPanel`), историю ходов (`GameHistoryPanel`), кнопки управления. `TournamentPageController` и `AlgorithmComparisonPageController` реализуют соответствующие режимы.

3.2. Взаимодействие с приложением

При запуске приложения пользователь попадает на стартовый экран, на котором настраиваются параметры предстоящей партии (рисунок 3).

Выбор размера поля. Доступны три варианта: малое поле 7×7 с 8 перегородками у каждого игрока, стандартное 9×9 с 10 перегородками, большое 11×11

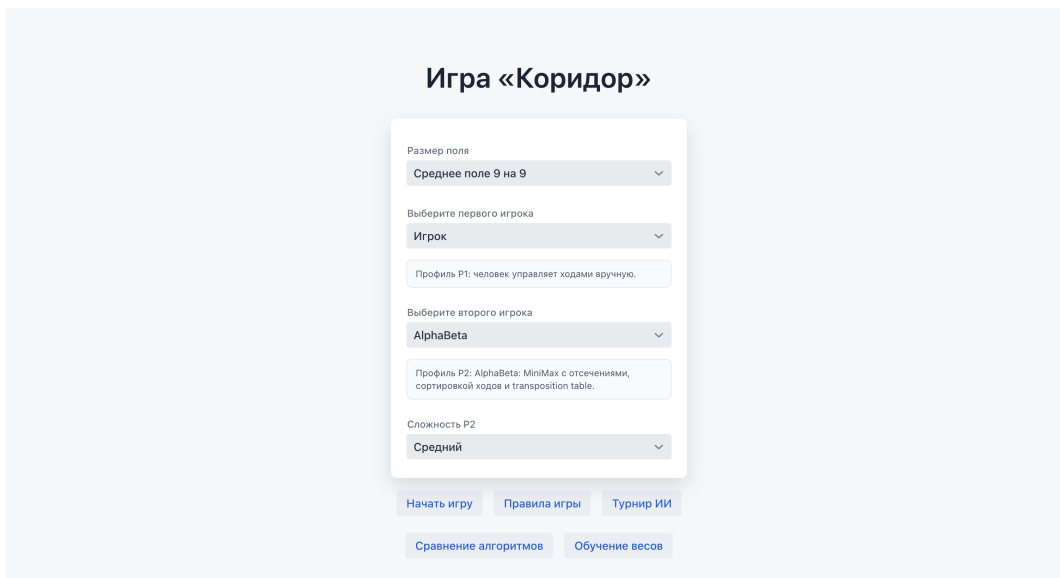


Рис. 3 — Стартовый экран выбора режима игры и алгоритмов

с 12 перегородками. Размер поля влияет на все параметры алгоритмов: лимиты времени и максимальные глубины масштабируются пропорционально.

Выбор первого игрока. Первым может управлять человек или один из четырёх алгоритмов ИИ. Если выбран ИИ, дополнительно настраивается его уровень сложности.

Выбор второго игрока. Второй игрок всегда управляется ИИ; выбирается алгоритм и уровень сложности. Карточки профилей отображают краткое описание выбранной стратегии.

На игровом экране пользователь видит несколько областей. Пример этого экрана приведён выше на рисунке 1.

Игровое поле отрисовывается в виде сетки: клетки (тайлы) чередуются с промежутками (щелями), в которых размещаются перегородки. Каждая клетка имеет размер 42–50 пикселей в зависимости от размера поля. Фишки отображаются цветными кружками (синий — P1, красный — P2). Допустимые для перемещения клетки подсвечиваются зелёным при наведении. При наведении на щель между клетками предварительно подсвечивается потенциальная перегородка.

Перемещение фишки выполняется кликом на целевую клетку. Если ход недопустим, появляется уведомление. Если ход совершается в режиме воспроизведения — он игнорируется.

Установка перегородки выполняется кликом на промежуток между клетками. `GameRules.extractWall` определяет по координатам рендера начальную и конечную клетки перегородки. Затем `GameRules.canPlaceWall` копирует доску, применяет перегородку и проверяет достижимость целевых линий обоих игроков через BFS.

Ход ИИ. В режиме «Игрок против ИИ» после хода пользователя автоматически запускается выбранный алгоритм. В режиме «ИИ против ИИ» каждый ход

требует нажатия кнопки «Сделать ход ИИ», что позволяет наблюдать за партией пошагово. Рядом с кнопкой отображается, чья очередь делать ход.

Подсказка. Кнопка «Подсказка хода» запускает все четыре алгоритма на текущей позиции и подсвечивает рекомендованные ими ходы разными цветами: синий (MiniMax), жёлтый (AlphaBeta), зелёный (MonteCarlo), розовый (Random). Если несколько алгоритмов рекомендуют один ход — клетка закрашивается градиентом.

Панель информации отображает: чья очередь; количество оставшихся перекрестков у каждого игрока; легенду цветов алгоритмов; отчёт о последнем ходе ИИ (оценка, время, глубина, узлы, отсечения, попадания в ТТ, текстовое пояснение); статистику партии (число ходов, время).

Режим воспроизведения. Кнопки «←», «→» и «К текущему» позволяют листать историю партии, просматривая состояние доски на любом из сыгранных ходов. История ходов дублируется текстовым списком в правой панели.

Турнирный режим позволяет задать два алгоритма, их уровни сложности, размер поля и число партий. Результаты выводятся в режиме реального времени: по завершении каждой партии обновляется счётчик побед. По завершении серии выводится сводная таблица. Пример работы режима показан на рисунке 4.

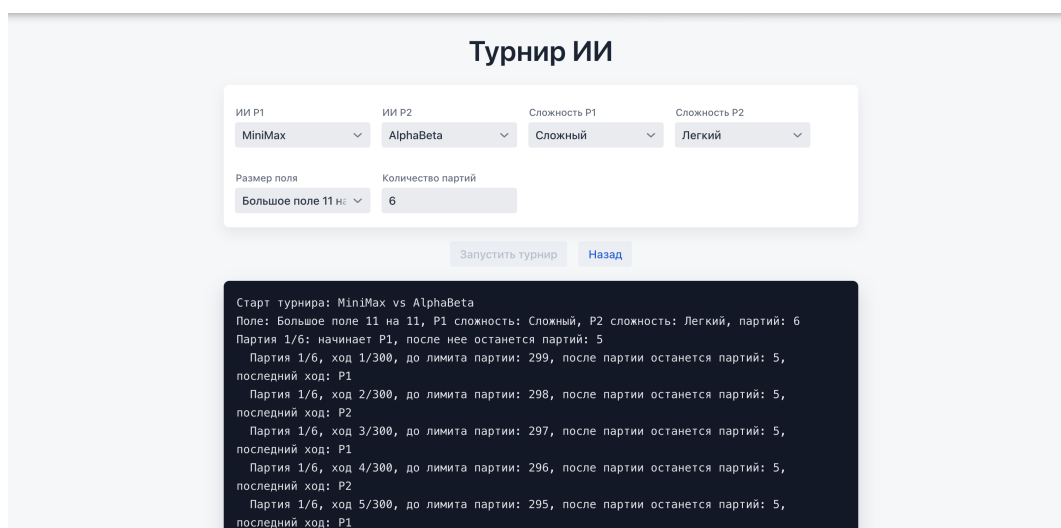


Рис. 4 — Экран турнирного режима ИИ

Режим сравнения алгоритмов автоматически проводит серию матчей между всеми попарными комбинациями четырёх алгоритмов (без зеркальных повторов) и выводит сводную таблицу с процентом побед, средним числом ходов, средним временем хода, средней глубиной поиска, числом узлов, отсечений и попаданий в ТТ. Интерфейс режима приведён на рисунке 5.

Режим обучения весов запускает серию self-play партий между двумя wybranymi алгоритмами. По завершении каждой партии сервис обучения обновляет веса функции оценки, сохраняет их в файл и выводит ход эксперимента в лог. Пример логов обучения показан на рисунке 6.

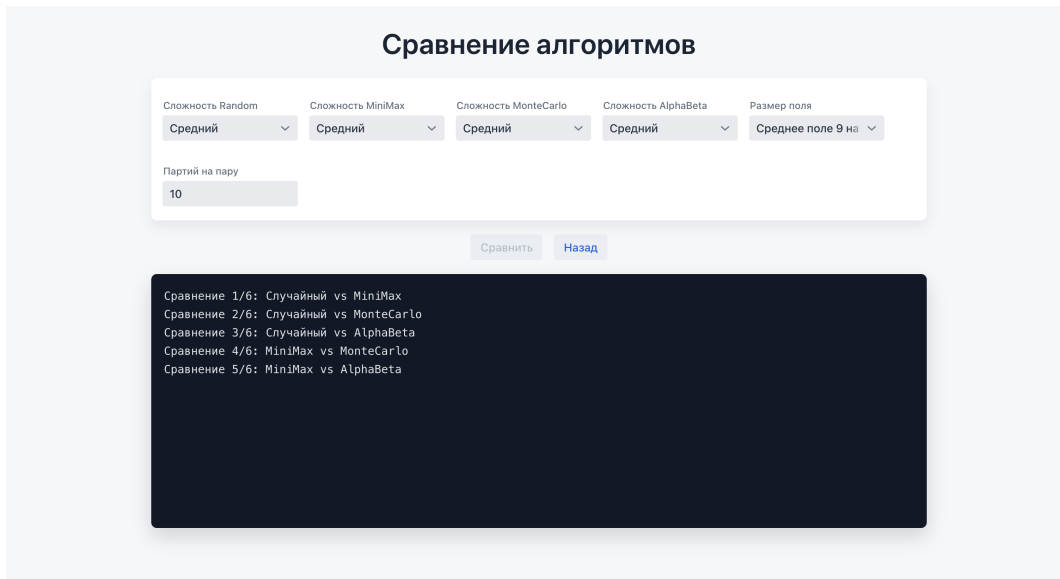


Рис. 5 — Экран сравнения алгоритмов

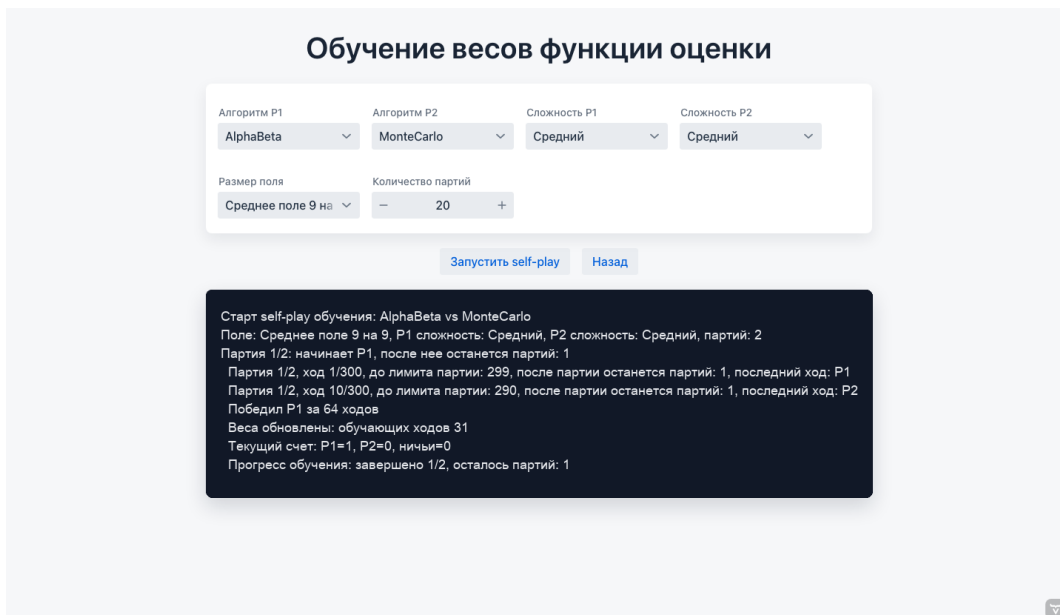


Рис. 6 — Экран self-play обучения весов функции оценки

4. Сравнение алгоритмов

4.1. Методика сравнения

Сравнение алгоритмов выполняется в отдельном режиме приложения «Сравнение алгоритмов». Цель режима — получить не единичный результат отдельной партии, а агрегированную оценку поведения стратегий на одинаковых условиях.

В эксперименте участвуют четыре алгоритма:

- RandomAlgorithm;
- MinimaxAlgorithm;
- AlphaBetaAlgorithm;
- MonteCarloAlgorithm.

Алгоритмы сравниваются попарно. Для исключения повторов рассматриваются только неупорядоченные пары: если матч MiniMax против Random уже проведён, обратная пара Random против MiniMax отдельно не запускается. При четырёх алгоритмах количество различных пар равно:

$$C_4^2 = \frac{4 \cdot 3}{2} = 6.$$

Внутри каждой пары проводится заданное пользователем число партий. Для компенсации преимущества первого хода роли игроков чередуются: часть партий алгоритм проводит за первого игрока, часть — за второго. Размер поля и уровень сложности каждого алгоритма задаются перед запуском эксперимента.

Важно, что в режиме сравнения алгоритмов обучение весов функции оценки не выполняется. Это сделано намеренно: если во время эксперимента функция оценки будет изменяться, более поздние партии будут проходить уже при других условиях, и сравнение потеряет воспроизводимость. Поэтому веса считаются фиксированными на протяжении всего запуска сравнения.

4.2. Собираемые метрики

Для анализа недостаточно знать только победителя. Два алгоритма могут иметь близкую долю побед, но существенно отличаться по времени работы, глубине поиска или количеству просмотренных состояний. Поэтому приложение собирает несколько групп метрик.

Метрики глубины, узлов, отсечений и попаданий в таблицу транспозиций особенно важны для сравнения MinimaxAlgorithm и AlphaBetaAlgorithm. Оба алгоритма основаны на дереве игры, но альфа-бета должен просматривать

Метрики сравнения алгоритмов

Метрика	Смысл
Количество партий	Общее число завершённых партий с участием алгоритма
Победы, поражения, ничьи	Основная игровая результативность алгоритма
Win rate	Доля побед алгоритма среди всех сыгранных им партий
Среднее число ходов	Косвенная характеристика стиля игры и сложности партий
Среднее время хода	Практическая вычислительная стоимость алгоритма
Средняя глубина	Насколько глубоко алгоритм успевает просмотреть дерево
Среднее число узлов	Объём реально просмотренного дерева поиска
Среднее число отсечений	Эффективность alpha-beta оптимизации
Попадания в ТТ	Эффективность таблицы транспозиций

меньше узлов при большей достижимой глубине. Для MonteCarloAlgorithm ключевыми являются число итераций и время хода, поскольку качество решения определяется количеством проведённых симуляций.

4.3. План эксперимента

Базовый эксперимент проводится на стандартном поле 9×9 . Для каждого алгоритма устанавливается уровень сложности MEDIUM. Каждая пара алгоритмов играет по 10 партий. Тогда суммарное число партий равно:

$$6 \cdot 10 = 60.$$

После базового эксперимента целесообразно провести дополнительный запуск на уровне HARD, поскольку именно на высоком уровне сложности сильнее проявляются различия между переборными алгоритмами. AlphaBeta получает возможность искать глубже за счёт отсечений, тогда как MiniMax ограничивается меньшей глубиной из-за экспоненциального роста дерева.

4.4. Форма представления результатов

Итоговая статистика может быть представлена в двух формах. Первая форма — попарная таблица побед, показывающая, как конкретные алгоритмы играют друг против друга.

Таблица 4.2

Попарные результаты сравнения алгоритмов

Алгоритм P1	Алгоритм P2	Игры	Победы P1	Победы P2	Ничьи	Ср. ходов	Ср. мс	Ср. глуб.	Ср. узлы	Ср. отсеч. / ТТ
Случайный	MiniMax	10	0	8	2	66,50	151,2	2,46	1088,5	0,0 / 0,0
Случайный	MonteCarlo	10	0	10	0	142,50	1051,7	31,67	140,8	0,0 / 0,0
Случайный	AlphaBeta	10	0	10	0	73,50	412,4	3,41	1307,5	249,2 / 198,9
MiniMax	MonteCarlo	10	4	6	0	98,30	1360,1	33,00	947,1	0,0 / 0,0
MiniMax	AlphaBeta	10	1	1	8	41,80	1215,3	4,85	4652,7	503,6 / 371,0
MonteCarlo	AlphaBeta	10	1	6	3	79,90	1839,7	34,28	1504,6	266,5 / 217,4

Вторая форма — агрегированная таблица по каждому алгоритму. Она удобнее для общего вывода, поскольку показывает не только победы, но и вычислительные затраты.

Таблица 4.3

Сводная статистика алгоритмов

Алгоритм	Партий	Победы	Поражения	Ничьи	Winrate	Очки/партия	Ср. ходов	Ср. мс	Ср. глуб.	Ср. узлы	Ср. отсеч. / ТТ
Случайный	30	0	28	2	0,00%	0,03	94,17	538,4	12,51	845,6	83,1 / 66,3
MiniMax	30	13	7	10	43,33%	0,60	68,87	908,9	13,44	2229,4	167,9 / 123,7
MonteCarlo	30	17	10	3	56,67%	0,62	106,90	1417,2	32,98	864,2	88,8 / 72,5
AlphaBeta	30	17	2	11	56,67%	0,75	65,07	1155,8	14,18	2488,3	339,8 / 262,4

Для AlphaBeta дополнительно анализируются среднее число отсечений и попаданий в таблицу транспозиций. Эти показатели позволяют подтвердить, что улучшение относительно MiniMax связано не только с увеличенным лимитом времени, но и с реальным сокращением числа просматриваемых ветвей.

4.5. Анализ результатов

Фактические результаты подтверждают, что случайный алгоритм является базовой линией: за 30 партий он не одержал ни одной победы и набрал только две ничьи. Это ожидаемо, поскольку даже при использовании функции оценки он не строит дерево последствий и не анализирует ответы соперника.

MiniMax заметно сильнее случайного алгоритма: он выиграл 13 партий из 30 и набрал 0,60 очка за партию. При этом его слабое место сохраняется — экспоненциальный рост числа состояний. Даже при ограничении ширины до 16 ходов глубина 5 даёт до $16^5 \approx 1,05 \cdot 10^6$ узлов в худшем случае, поэтому алгоритм вынужден жёстко ограничивать глубину и ширину перебора.

AlphaBeta показал лучший интегральный результат: 17 побед, только 2 поражения и 0,75 очка за партию. При равном winrate с MonteCarlo он имеет меньше поражений и больше ничьих, что говорит о более устойчивом поведении в сложных позициях. Среднее число отсечений 339,8 и 262,4 попадания в таблицу транспозиций за ход подтверждают, что преимущество алгоритма связано не только с перебором, но и с эффективным сокращением дерева поиска.

MonteCarlo также показал высокий результат — 17 побед из 30 и winrate 56,67%. Его партии в среднем длиннее, чем у AlphaBeta, а среднее время хода

выше. Это отражает природу метода: качество решения зависит от числа симуляций и устойчивости rollout-стратегии. В игре «Коридор» это особенно важно из-за большого числа возможных стен: если бюджет итераций мал, статистика по отдельным ходам может быть недостаточно устойчивой.

4.6. Ограничения эксперимента

При интерпретации результатов необходимо учитывать несколько ограничений.

Во-первых, алгоритмы используют разные типы вычислительного бюджета: MiniMax и AlphaBeta ограничены глубиной и временем, а MCTS — временем, числом итераций и глубиной симуляции. Поэтому сравнение не является сравнением «одинаковых» алгоритмов при одинаковой глубине, а является сравнением реализованных игровых стратегий в условиях приложения.

Во-вторых, результат зависит от текущих весов функции оценки. Для воспроизводимости в тексте работы необходимо указать значения весов из файла `data/evaluation-weights.properties`, использованные перед запуском эксперимента.

В-третьих, выбор числа партий влияет на статистическую устойчивость результатов. Серия из 60 партий подходит для первичного сравнения, но для более надёжного вывода желательно увеличить число партий в каждой паре.

Заключение

В ходе выполнения выпускной квалификационной работы была исследована настольная логическая игра «Коридор» и построена её графовая модель. Реализованы алгоритмы проверки корректности перемещения фишки с учётом всех правил прыжков через соперника и проверки корректности установки перегородки через обход графа в ширину. Реализован алгоритм поиска кратчайшего пути до целевой линии на основе BFS с восстановлением пути.

Разработана многофакторная функция оценки позиции, учитывающая семь признаков: разность длин путей, мобильность фишек, преимущество в перегородках, продвижение к цели и эндшпильные бонусы. Реализован механизм коррекции весов функции оценки по результатам завершённых партий с сохранением в файл конфигурации между сессиями.

Реализованы четыре алгоритма программного соперника с тремя уровнями сложности каждый. Случайный алгоритм служит базовой линией. Минимакс с кэшированием и ограничением ширины перебора обеспечивает разумный баланс качества и скорости. Минимакс с альфа-бета отсечением, сортировкой ходов, таблицей транспозиций с типами границ и затухающим поиском в шумных позициях является наиболее сильным из переборных алгоритмов. Метод Монте-Карло с UCT, эвристическим выбором ходов в симуляциях и условием досрочного останова обеспечивает альтернативный подход, основанный на статистике симуляций.

Разработано игровое приложение с режимами «Игрок против ИИ», «ИИ против ИИ», режимом подсказок одновременно от всех четырёх алгоритмов, режимом воспроизведения партии, турнирным режимом и режимом сравнения алгоритмов. После проведения экспериментального запуска таблицы сравнения алгоритмов должны быть заполнены фактическими результатами.

Список литературы

- [1] Gigamic. Правила игры Коридор [Электронный ресурс]. — Режим доступа: <https://en.gigamic.com/modern-classics/107-quoridor.html> (дата обращения: 29.04.2026).
- [2] BoardGameGeek. Коридор [Электронный ресурс]. — Режим доступа: <https://boardgamegeek.com/boardgame/624/quoridor> (дата обращения: 29.04.2026).
- [3] BoardGameGeek. Games 100 [Electronic resource]. — Режим доступа: https://boardgamegeek.com/wiki/page/Games_100 (online; accessed: 02.04.2026).
- [4] Games, Hobby. Коридор. Правила игры [Электронный ресурс]. — Режим доступа: https://hobbygames.ru/download/rules/Quoridor_Rules.pdf (дата обращения: 03.04.2026).
- [5] Mertens, J. A Koridor-playing agent [Text] / J. Mertens // Technical report. — 2006.
- [6] Hart, P. E. A Formal Basis for the Heuristic Determination of Minimum Cost Paths [Text] / P. E. Hart, N. J. Nilsson, B. Raphael // IEEE Transactions on Systems Science and Cybernetics. — 1968. — Vol. 4, no. 2. — P. 100–107.
- [7] Russell, S. Artificial Intelligence: A Modern Approach [Text] / S. Russell, P. Norvig. — Hoboken : Pearson, 2021.
- [8] Coulom, R. Efficient Selectivity and Backup Operators in Monte-Carlo Tree Search [Text] / R. Coulom // Computers and Games. — 2006. — P. 72–83.
- [9] Browne, C. Connection Games: Variations on a Theme [Text] / C. Browne. — Wellesley : A K Peters, 2009.
- [10] Vaadin. Vaadin Flow Documentation [Electronic resource]. — Режим доступа: <https://vaadin.com/docs> (online; accessed: 29.04.2026).

Реализация интерфейса алгоритма

Листинг А.1 — Интерфейс Algorithm

```

1 public interface Algorithm {
2     int WIN_SCORE = 100_000;
3
4     ComputerAlgorithmType getType();
5     Move getMove(Board board, ComputerPlayerHardnessLevel
        hardnessLevel, int playerId, int wallsLeft1, int
        wallsLeft2);
6
7     default AlgorithmReport getLastReport() {
8         return null;
9     }
10
11     default void setRecentPositions(List<Position>
        recentPositions) {
12     }
13
14     default int movementPreference(Move move, Board board, int
        playerId, int size, List<Position> recentPositions) {
15         if (move.getMoveType() != MoveType.MOVE_PLAYER || move.
            getPlayerId() != playerId) {
16             return 0;
17         }
18
19         int score = 0;
20         Position current = getCurrentPosition(board, playerId);
21         Position target = move.getEndPosition();
22         int direction = playerId == 1 ? -1 : 1;
23         int rowDelta = target.row() - current.row();
24
25         if (rowDelta == direction) {
26             score += 45;
27         } else if (rowDelta == -direction) {
28             score -= 120;
29         }

```

```

30
31     for (int index = 0; index < recentPositions.size(); ++
32         index) {
33         Position recent = recentPositions.get(index);
34         if (recent.equals(current)) {
35             continue;
36         }
37         if (target.equals(recent)) {
38             score -= index <= 1 ? 500 : 180;
39         }
40     }
41
42     int before = Math.abs(current.row() - getTargetRow(
43         playerId, size));
44     int after = Math.abs(target.row() - getTargetRow(
45         playerId, size));
46     score += (before - after) * 35;
47     return score;
48 }
49
50 default List<Move> getMoves(Board board, int playerId, int
51     wallsLeft) {
52     List<Move> moves = new ArrayList<>();
53     Position pos = getCurrentPosition(board, playerId);
54
55     for (Position next : board.getAvailableMoves(pos)) {
56         moves.add(Move.movePlayer(playerId, next));
57     }
58
59     if (wallsLeft <= 0) {
60         return moves;
61     }
62
63     int size = (int) Math.sqrt(board.getTiles().size());
64     Set<String> used = new HashSet<>();
65     List<Position> enemyPath = getShortestPathCells(board,
66         getCurrentPosition(board, 3 - playerId),
67         getTargetRow(3 - playerId, size));
68     List<Position> myPath = getShortestPathCells(board,
69         getCurrentPosition(board, playerId),

```

```

64         getTargetRow(playerId, size));
65     List<Position> candidateCells = new ArrayList<>();
66     List<Position> enemyFront = enemyPath.subList(0, Math.
        min(12, enemyPath.size()));
67     List<Position> myFront = myPath.subList(0, Math.min(7,
        myPath.size()));
68     candidateCells.addAll(enemyFront);
69     candidateCells.addAll(myFront);
70     addNeighborhood(candidateCells, getCurrentPosition(
        board, 3 - playerId), size);
71     for (Position cell : enemyFront.subList(0, Math.min(5,
        enemyFront.size())) {
72         addNeighborhood(candidateCells, cell, size, 1);
73     }
74
75     int beforeEnemy = calculateShortestPath(board,
        getCurrentPosition(board, 3 - playerId),
        getTargetRow(3 - playerId, size));
76     int beforeMine = calculateShortestPath(board,
        getCurrentPosition(board, playerId), getTargetRow(
        playerId, size));
77     List<Move> wallMoves = new ArrayList<>();
78
79     for (Position cell : candidateCells) {
80         int i = cell.row();
81         int j = cell.col();
82
83         if (i < size - 1) {
84             Position start = new Position(i, j);
85             Position end = new Position(i + 1, j);
86             String wallKey = start + "-" + end;
87
88             if (used.add(wallKey) && isWallPlaceable(board,
                start, end) && isValidWall(board, start,
                end)) {
89                 if (isStrategicWall(board, start, end,
                    playerId, size, beforeEnemy, beforeMine)
                    ) {
90                     wallMoves.add(Move.placeWall(playerId,
                        start, end));

```

```

91         }
92     }
93 }
94
95     if (j < size - 1) {
96         Position start = new Position(i, j);
97         Position end = new Position(i, j + 1);
98         String wallKey = start + "-" + end;
99
100         if (used.add(wallKey) && isWallPlaceable(board,
101             start, end) && isValidWall(board, start,
102             end)) {
103             if (isStrategicWall(board, start, end,
104                 playerId, size, beforeEnemy, beforeMine)
105                 ) {
106                 wallMoves.add(Move.placeWall(playerId,
107                     start, end));
108             }
109         }
110     }
111     wallMoves.sort((a, b) -> Integer.compare(
112         wallImpact(board, b, playerId, size),
113         wallImpact(board, a, playerId, size)
114     ));
115     moves.addAll(wallMoves);
116     return moves;
117 }
118
119 default void addNeighborhood(List<Position> cells, Position
120     center, int size) {
121     addNeighborhood(cells, center, size, 2);
122 }
123
124 default void addNeighborhood(List<Position> cells, Position
125     center, int size, int radius) {
126     for (int di = -radius; di <= radius; ++di) {
127         for (int dj = -radius; dj <= radius; ++dj) {
128             int nextRow = center.row() + di;
129             int nextCol = center.col() + dj;

```

```

124         if (nextRow >= 0 && nextCol >= 0 && nextRow <
            size && nextCol < size) {
125             cells.add(new Position(nextRow, nextCol));
126         }
127     }
128 }
129 }
130
131 default boolean isStrategicWall(Board board, Position start
    , Position end, int playerId, int size,
132                               int beforeEnemy, int
                                beforeMine) {
133     Board copy = board.copy();
134     copy.placeWall(start, end);
135
136     int afterEnemy = calculateShortestPath(copy,
        getCurrentPosition(copy, 3 - playerId), getTargetRow(
        3 - playerId, size));
137     int afterMine = calculateShortestPath(copy,
        getCurrentPosition(copy, playerId), getTargetRow(
        playerId, size));
138
139     int enemyGain = afterEnemy - beforeEnemy;
140     int myCost = Math.max(0, afterMine - beforeMine);
141
142     if (enemyGain >= 2) {
143         return true;
144     }
145     if (beforeEnemy <= 3 && enemyGain > 0) {
146         return true;
147     }
148     return enemyGain > 0 && enemyGain >= myCost;
149 }
150
151 default boolean isWallPlaceable(Board board, Position start
    , Position end) {
152     try {
153         Board copy = board.copy();
154         copy.placeWall(start, end);
155         return true;

```

```

156         } catch (Exception e) {
157             return false;
158         }
159     }
160
161     default List<Position> getShortestPathCells(Board board,
162         Position start, int targetRow) {
163         Map<Position, Position> parent = new HashMap<>();
164         Queue<Position> queue = new ArrayDeque<>();
165         Set<Position> visited = new HashSet<>();
166
167         queue.add(start);
168         visited.add(start);
169
170         Position end = null;
171         while (!queue.isEmpty()) {
172             Position curr = queue.poll();
173
174             if (curr.row() == targetRow) {
175                 end = curr;
176                 break;
177             }
178
179             for (Position next : board.getPathNeighbors(curr))
180             {
181                 if (visited.add(next)) {
182                     parent.put(next, curr);
183                     queue.add(next);
184                 }
185             }
186
187             List<Position> path = new ArrayList<>();
188             while (end != null) {
189                 path.add(end);
190                 end = parent.get(end);
191             }
192             Collections.reverse(path);
193             return path;
194         }

```

```

194
195     default boolean isValidWall(Board board, Position start,
196         Position end) {
197         try {
198             Board copy = board.copy();
199             copy.placeWall(start, end);
200
201             int size = (int) Math.sqrt(board.getTiles().size())
202                 ;
203
204             return hasPath(copy, copy.getPositionOfPlayer1(),
205                 0) && hasPath(copy, copy.getPositionOfPlayer2(),
206                 size - 1);
207         } catch (Exception e) {
208             return false;
209         }
210     }
211
212     private boolean hasPath(Board board, Position start, int
213         targetRow) {
214         Set<Position> visited = new HashSet<>();
215         Queue<Position> queue = new ArrayDeque<>();
216
217         queue.add(start);
218         visited.add(start);
219
220         while (!queue.isEmpty()) {
221             Position curr = queue.poll();
222
223             if (curr.row() == targetRow) {
224                 return true;
225             }
226
227             for (Position next : board.getPathNeighbors(curr))
228                 {
229                 if (visited.add(next)) {
230                     queue.add(next);
231                 }
232             }
233         }
234     }

```

```

228         return false;
229     }
230
231     default void applyMove(Board board, Move move) {
232         if (move.getMoveType() == MoveType.MOVE_PLAYER) {
233             if (move.getPlayerId() == 1) {
234                 board.setPositionOfPlayer1(move.getEndPosition
235                     ());
236             } else {
237                 board.setPositionOfPlayer2(move.getEndPosition
238                     ());
239             }
240         } else {
241             board.placeWall(move.getStartPosition(), move.
242                 getEndPosition());
243         }
244     }
245
246     default Position getCurrentPosition(Board board, int
247         playerId) {
248         return playerId == 1 ? board.getPositionOfPlayer1() :
249             board.getPositionOfPlayer2();
250     }
251
252     default int getTargetRow(int playerId, int size) {
253         return playerId == 1 ? 0 : size - 1;
254     }
255
256     default boolean isWin(Board board, int playerId, int size)
257     {
258         return getCurrentPosition(board, playerId).row() ==
259             getTargetRow(playerId, size);
260     }
261
262     default Integer terminalScore(Board board, int rootPlayerId
263         , int size, int depth) {
264         if (isWin(board, rootPlayerId, size)) {
265             return WIN_SCORE + depth;
266         }
267         if (isWin(board, 3 - rootPlayerId, size)) {

```

```

260         return -WIN_SCORE - depth;
261     }
262     return null;
263 }
264
265 default Move findEndgameMove(Board board, int playerId, int
    wallsLeft) {
266     int size = (int) Math.sqrt(board.getTiles().size());
267     Position current = getCurrentPosition(board, playerId);
268     for (Position next : board.getAvailableMoves(current))
269     {
270         if (next.row() == getTargetRow(playerId, size)) {
271             return Move.movePlayer(playerId, next);
272         }
273     }
274
275     if (wallsLeft <= 0) {
276         return null;
277     }
278
279     int enemy = 3 - playerId;
280     int enemyDistance = calculateShortestPath(board,
        getCurrentPosition(board, enemy), getTargetRow(enemy
        , size));
281
282     if (enemyDistance > 2) {
283         return null;
284     }
285
286     Move bestWall = null;
287     int bestImpact = 0;
288     for (Move move : getMoves(board, playerId, wallsLeft))
289     {
290         if (move.getMoveType() != MoveType.PLACE_WALL) {
291             continue;
292         }
293         int impact = wallImpact(board, move, playerId, size
        );
294         if (impact > bestImpact) {
295             bestImpact = impact;
296             bestWall = move;
297         }
298     }
299     return bestWall;
300 }

```

```

294         }
295     }
296     return bestImpact > 0 ? bestWall : null;
297 }
298
299 default boolean isNoisyPosition(Board board, int playerId,
300     int size) {
301     int myDist = calculateShortestPath(board,
302         getCurrentPosition(board, playerId), getTargetRow(
303             playerId, size));
304     int enemyDist = calculateShortestPath(board,
305         getCurrentPosition(board, 3 - playerId),
306         getTargetRow(3 - playerId, size));
307     return myDist <= 2 || enemyDist <= 2;
308 }
309
310 default List<Move> getQuiescenceMoves(Board board, int
311     currentPlayer, int wallsLeft, int ignoredRootPlayerId,
312     int size) {
313     List<Move> moves = new ArrayList<>();
314     Position current = getCurrentPosition(board,
315         currentPlayer);
316     for (Position next : board.getAvailableMoves(current))
317     {
318         if (next.row() == getTargetRow(currentPlayer, size)
319             ) {
320             moves.add(Move.movePlayer(currentPlayer, next))
321                 ;
322         }
323     }
324
325     if (wallsLeft > 0) {
326         List<Move> tacticalWalls = getMoves(board,
327             currentPlayer, wallsLeft).stream()
328             .filter(move -> move.getMoveType() ==
329                 MoveType.PLACE_WALL)
330             .sorted((a, b) -> Integer.compare(
331                 wallImpact(board, b, currentPlayer,
332                     size),
333                 wallImpact(board, a, currentPlayer,

```

```

size)
320         ))
321         .limit(8)
322         .toList();
323         moves.addAll(tacticalWalls);
324     }
325     return moves;
326 }
327
328 default int wallImpact(Board board, Move move, int playerId
    , int size) {
329     int enemy = 3 - playerId;
330     int beforeEnemy = calculateShortestPath(board,
        getCurrentPosition(board, enemy), getTargetRow(enemy
        , size));
331     int beforeMine = calculateShortestPath(board,
        getCurrentPosition(board, playerId), getTargetRow(
        playerId, size));
332     Board copy = board.copy();
333     applyMove(copy, move);
334     int afterEnemy = calculateShortestPath(copy,
        getCurrentPosition(copy, enemy), getTargetRow(enemy,
        size));
335     int afterMine = calculateShortestPath(copy,
        getCurrentPosition(copy, playerId), getTargetRow(
        playerId, size));
336     return (afterEnemy - beforeEnemy) * 100 - Math.max(0,
        afterMine - beforeMine) * 45;
337 }
338
339 default int evaluate(Board board, int playerId, int size,
    int wallsLeft1, int wallsLeft2) {
340     Integer terminal = terminalScore(board, playerId, size,
        0);
341     if (terminal != null) {
342         return terminal;
343     }
344
345     return evaluationFeatures(board, playerId, size,
        wallsLeft1, wallsLeft2)

```

```

346         .score(EvaluationWeightsStore.current());
347     }
348
349     default EvaluationFeatures evaluationFeatures(Board board,
        int playerId, int size, int wallsLeft1, int wallsLeft2)
    {
350         int myDist = calculateShortestPath(board,
            getCurrentPosition(board, playerId), getTargetRow(
                playerId, size));
351         int enemyDist = calculateShortestPath(board,
            getCurrentPosition(board, 3 - playerId),
            getTargetRow(3 - playerId, size));
352
353         int myWalls = playerId == 1 ? wallsLeft1 : wallsLeft2;
354         int enemyWalls = playerId == 1 ? wallsLeft2 :
            wallsLeft1;
355
356         int myRow = getCurrentPosition(board, playerId).row();
357         int enemyRow = getCurrentPosition(board, 3 - playerId).
            row();
358         int myProgress = playerId == 1 ? size - 1 - myRow :
            myRow;
359         int enemyProgress = playerId == 1 ? enemyRow : size - 1
            - enemyRow;
360
361         return new EvaluationFeatures(
362             enemyDist - myDist,
363             board.getAvailableMoves(getCurrentPosition(
                board, playerId)).size(),
364             -board.getAvailableMoves(getCurrentPosition(
                board, 3 - playerId)).size(),
365             myWalls - enemyWalls,
366             myProgress - enemyProgress,
367             endgameFeature(myDist),
368             -endgameFeature(enemyDist)
369         );
370     }
371
372     default EvaluationLearningSample buildLearningSample(Board
        before, Move move, int playerId,

```

```

373                                     int
                                         size
                                         ,
                                         int

                                         wallsLeft1
                                         ,
                                         int

                                         wallsLeft2
                                         ) {
374     if (move == null) {
375         return null;
376     }
377
378     EvaluationFeatures beforeFeatures = evaluationFeatures(
        before, playerId, size, wallsLeft1, wallsLeft2);
379     Board after = before.copy();
380     applyMove(after, move);
381
382     int newWallsLeft1 = wallsLeft1;
383     int newWallsLeft2 = wallsLeft2;
384     if (move.getMoveType() == MoveType.PLACE_WALL) {
385         if (move.getPlayerId() == 1) {
386             --newWallsLeft1;
387         } else {
388             --newWallsLeft2;
389         }
390     }
391
392     EvaluationFeatures afterFeatures = evaluationFeatures(
        after, playerId, size, newWallsLeft1, newWallsLeft2)
        ;
393     return new EvaluationLearningSample(playerId,
        beforeFeatures, afterFeatures);
394 }
395
396 default int endgameFeature(int myDist) {
397     if (myDist <= 1) {
398         return 10;

```

```

399         }
400         if (myDist == 2) {
401             return 4;
402         }
403         if (myDist == 3) {
404             return 1;
405         }
406         return 0;
407     }
408
409     default int calculateShortestPath(Board board, Position
start, int targetRow) {
410         Set<Position> visited = new HashSet<>();
411         Queue<Position> queue = new ArrayDeque<>();
412
413         queue.add(start);
414         visited.add(start);
415
416         int depth = 0;
417
418         while (!queue.isEmpty()) {
419             for (int k = queue.size(); k > 0; --k) {
420                 Position curr = queue.poll();
421
422                 if (curr.row() == targetRow) return depth;
423
424                 for (Position next : board.getPathNeighbors(
curr)) {
425                     if (visited.add(next)) {
426                         queue.add(next);
427                     }
428                 }
429             }
430             depth++;
431         }
432         return 1000;
433     }
434
435     default long getTimeLimit(ComputerPlayerHardnessLevel level
, int size) {

```

```

436         int multiplier = Math.max(1, size / GameSize.NORMAL.
            getAmountOfTilesPerSide());
437     return switch (level) {
438         case EASY -> 700L * multiplier;
439         case MEDIUM -> 1600L * multiplier;
440         case HARD -> 2800L * multiplier;
441     };
442 }
443
444 default String boardKey(Board board) {
445     StringBuilder key = new StringBuilder();
446     key.append(board.getPositionOfPlayer1()).append('/').
        append(board.getPositionOfPlayer2()).append('/');
447
448     board.getTiles().entrySet().stream()
449         .sorted(Comparator
450             .comparingInt((Map.Entry<Position, ?>
451                 entry) -> entry.getKey().row())
452             .thenComparingInt(entry -> entry.getKey()
453                 ().col()))
454         .forEach(entry -> {
455             key.append(entry.getKey());
456             key.append(entry.getValue().
457                 isLeftMovementAvailable() ? '1' : '0');
458             key.append(entry.getValue().
459                 isForwardMovementAvailable() ? '1' : '0'
460                 );
461             key.append(entry.getValue().
462                 isRightMovementAvailable() ? '1' : '0');
463             key.append(entry.getValue().
464                 isBackwardsMovementAvailable() ? '1' : '
0');
465         });
466
467     return key.toString();
468 }

```

Реализация случайного алгоритма

Листинг Б.1 — Класс RandomAlgorithm

```

1 public class RandomAlgorithm implements Algorithm {
2     private static final Random random = new Random();
3     private AlgorithmReport lastReport;
4     private List<Position> recentPositions = List.of();
5
6     @Override
7     public ComputerAlgorithmType getType() {
8         return ComputerAlgorithmType.RANDOM;
9     }
10
11    @Override
12    public AlgorithmReport getLastReport() {
13        return lastReport;
14    }
15
16    @Override
17    public void setRecentPositions(List<Position>
18        recentPositions) {
19        this.recentPositions = recentPositions == null ? List.
20            of() : List.copyOf(recentPositions);
21    }
22
23    @Override
24    public Move getMove(Board board,
25        ComputerPlayerHardnessLevel hardnessLevel, int playerId,
26        int wallsLeft1, int wallsLeft2) {
27        long startedAt = System.currentTimeMillis();
28        int amountOfWallsLeft = playerId == 1 ? wallsLeft1 :
29            wallsLeft2;
30        List<Move> moves = getMoves(board, playerId,
31            amountOfWallsLeft);
32
33        if (moves.isEmpty()) return null;
34    }
35 }

```

```

29     Move tacticalMove = findEndgameMove(board, playerId,
        amountOfWallsLeft);
30     if (tacticalMove != null) {
31         lastReport = new AlgorithmReport(getType().
            getDescription(), tacticalMove,
32             evaluateAfterMove(board, tacticalMove,
                playerId, wallsLeft1, wallsLeft2),
33             0, 1, 1, 0, 0,
34             System.currentTimeMillis() - startedAt,
35             "Случайный_алгоритм_применил_эндшпильное_пр
                авило_перед_случайным_выбором");
36         return tacticalMove;
37     }
38
39     Move result;
40     switch (hardnessLevel) {
41         case MEDIUM -> {
42             moves.sort((a, b) -> Integer.compare(
43                 evaluateAfterMove(board, b, playerId,
44                     wallsLeft1, wallsLeft2),
45                 evaluateAfterMove(board, a, playerId,
46                     wallsLeft1, wallsLeft2)
47             ));
48             int half = moves.size() / 2 + 1;
49             result = moves.get(random.nextInt(half));
50         }
51         case HARD -> {
52             moves.sort((a, b) -> Integer.compare(
53                 evaluateAfterMove(board, b, playerId,
54                     wallsLeft1, wallsLeft2),
55                 evaluateAfterMove(board, a, playerId,
56                     wallsLeft1, wallsLeft2)
57             ));
58             int top = Math.min(3, moves.size());
59             result = moves.get(random.nextInt(top));
60         }
61         default -> result = moves.get(random.nextInt(moves.
            size()));
62     }
63     lastReport = new AlgorithmReport(getType().

```

```

        getDescription(), result,
60         evaluateAfterMove(board, result, playerId,
            wallsLeft1, wallsLeft2),
61         1, moves.size(), moves.size(), 0, 0,
62         System.currentTimeMillis() - startedAt,
63         describeHardness(hardnessLevel));
64     return result;
65 }
66
67 private String describeHardness(ComputerPlayerHardnessLevel
    hardnessLevel) {
68     return switch (hardnessLevel) {
69         case EASY -> "Случайный_выбор_среди_всех_допустимых
            _ходов";
70         case MEDIUM -> "Случайный_выбор_из_верхней_половины
            _ходов_по_функции_оценки";
71         case HARD -> "Случайный_выбор_из_трех_лучших_ходов_
            по_функции_оценки";
72     };
73 }
74
75 private int evaluateAfterMove(Board board, Move move, int
    playerId, int wallsLeft1, int wallsLeft2) {
76     Board copy = board.copy();
77     applyMove(copy, move);
78     int size = (int) Math.sqrt(copy.getTiles().size());
79     int newWallsLeft1 = wallsLeft1;
80     int newWallsLeft2 = wallsLeft2;
81     if (move.getMoveType() == MoveType.PLACE_WALL) {
82         if (move.getPlayerId() == 1) {
83             --newWallsLeft1;
84         } else {
85             --newWallsLeft2;
86         }
87     }
88     return evaluate(copy, playerId, size, newWallsLeft1,
        newWallsLeft2)
89         + movementPreference(move, board, playerId,
            size, recentPositions);
90 }

```


Реализация алгоритма MiniMax

Листинг В.1 — Класс MinimaxAlgorithm

```

1 public class MinimaxAlgorithm implements Algorithm {
2     /**
3      * уже посчитанные ранее позиции
4      */
5     private final Map<String, Integer> cache = new HashMap<>();
6     private AlgorithmReport lastReport;
7     private long nodesVisited;
8     private long consideredMoves;
9     private List<Position> recentPositions = List.of();
10
11     @Override
12     public ComputerAlgorithmType getType() {
13         return ComputerAlgorithmType.MINIMAX;
14     }
15
16     @Override
17     public AlgorithmReport getLastReport() {
18         return lastReport;
19     }
20
21     @Override
22     public void setRecentPositions(List<Position>
23         recentPositions) {
24         this.recentPositions = recentPositions == null ? List.
25             of() : List.copyOf(recentPositions);
26     }
27
28     /**
29      * пока есть время, итеративно увеличиваем глубину поиска р
30      * ешения
31      * если время кончилось -> отдаем лучшее, что нашли
32      * MiniMax оставлен как честный перебор без сортировки и от
33      * сечений
34      */

```

```

31  @Override
32  public Move getMove(Board board,
    ComputerPlayerHardnessLevel hardnessLevel, int playerId,
    int wallsLeft1, int wallsLeft2) {
33      long startedAt = System.currentTimeMillis();
34      int size = (int) Math.sqrt(board.getTiles().size());
35      long timeLimit = getTimeLimit(hardnessLevel, size);
36      long endTime = System.currentTimeMillis() + timeLimit;
37      int maxDepth = getMaxDepth(hardnessLevel, size);
38
39      cache.clear();
40      nodesVisited = 0;
41      consideredMoves = 0;
42
43      int currentWalls = playerId == 1 ? wallsLeft1 :
        wallsLeft2;
44      Move tacticalMove = findEndgameMove(board, playerId,
        currentWalls);
45      if (tacticalMove != null) {
46          int tacticalScore = scoreMoveAfterApply(board,
            tacticalMove, playerId, size, wallsLeft1,
            wallsLeft2);
47          lastReport = new AlgorithmReport(
48              getType().getDescription(),
49              tacticalMove,
50              tacticalScore,
51              0,
52              nodesVisited,
53              1,
54              0,
55              0,
56              System.currentTimeMillis() - startedAt,
57              "MiniMax_применил_эндшпильное_правило:_неме
                дленная_победа_или_срочная_блокировка"
58          );
59          return tacticalMove;
60      }
61
62      Move best = null;
63      int bestScore = 0;

```

```

64     int reachedDepth = 0;
65     for (int depth = 1; depth <= maxDepth; depth++) {
66         if (System.currentTimeMillis() > endTime) {
67             break;
68         }
69         SearchResult result = search(board, depth, playerId
70             , size, wallsLeft1, wallsLeft2, endTime);
71         if (result.move() != null) {
72             best = result.move();
73             bestScore = result.score();
74             reachedDepth = depth;
75         }
76     }
77     lastReport = new AlgorithmReport(
78         getType().getDescription(),
79         best,
80         bestScore,
81         reachedDepth,
82         nodesVisited,
83         consideredMoves,
84         0,
85         0,
86         System.currentTimeMillis() - startedAt,
87         "MiniMax_перебрал_дерево_без_alpha-beta_отсечен
            ий"
            + " ;_лимит_времени:_ " + timeLimit + "_м
            с"
88     );
89     return best;
90 }
91
92 private int getMaxDepth(ComputerPlayerHardnessLevel level,
93     int size) {
94     boolean smallBoard = size <= GameSize.SMALL.
95         getAmountOfTilesPerSide();
96     boolean largeBoard = size > GameSize.NORMAL.
97         getAmountOfTilesPerSide();
98     return switch (level) {
99         case EASY -> 3;
100        case MEDIUM -> smallBoard ? 5 : 4;

```

```

98         case HARD -> largeBoard ? 4 : 5;
99     };
100 }
101
102 @Override
103 public long getTimeLimit(ComputerPlayerHardnessLevel level,
104     int size) {
105     boolean largeBoard = size > GameSize.NORMAL.
106         getAmountOfTilesPerSide();
107     return switch (level) {
108         case EASY -> largeBoard ? 900L : 700L;
109         case MEDIUM -> largeBoard ? 2200L : 1800L;
110         case HARD -> largeBoard ? 4200L : 3600L;
111     };
112 }
113
114 /**
115  * генерируем всевозможные ходы, отбираем из них ограниченн
116    ое количество
117  * потом прогоняем через minimax функцию оценки
118  */
119
120 private SearchResult search(Board board, int depth, int
121     playerId, int size,
122     int wallsLeft1, int wallsLeft2,
123     long endTime) {
124     List<Move> moves = limitMoves(getMoves(board, playerId,
125         wallsLeft1));
126     consideredMoves += moves.size();
127
128     Move bestMove = null;
129     int bestValue = Integer.MIN_VALUE;
130     for (Move move : moves) {
131         if (System.currentTimeMillis() > endTime) {
132             return new SearchResult(bestMove, bestValue);
133         }
134
135         Board copy = board.copy();
136         applyMove(copy, move);
137
138         int newWallsLeft1 = wallsLeft1, newWallsLeft2 =

```

```

        wallsLeft2;
132     if (move.getMoveType() == MoveType.PLACE_WALL) {
133         if (playerId == 1) {
134             --newWallsLeft1;
135         } else {
136             --newWallsLeft2;
137         }
138     }
139
140     int value = minimax(copy, depth - 1, false,
141         playerId, size, newWallsLeft1,
142         newWallsLeft2, endTime);
143     value += movementPreference(move, board, playerId,
144         size, recentPositions);
145     if (value > bestValue) {
146         bestValue = value;
147         bestMove = move;
148     }
149 }
150
151 /**
152  * если max = true (наш ход) - максимизируем оценку
153  * если max = false (ход противника) - минимизируем
154  */
155 private int minimax(Board board, int depth, boolean
156     maximizing,
157     int playerId, int size,
158     int wallsLeft1, int wallsLeft2, long
159     endTime) {
160     nodesVisited++;
161     if (System.currentTimeMillis() > endTime) {
162         return evaluate(board, playerId, size, wallsLeft1,
163             wallsLeft2);
164     }
165     String key = boardKey(board) + "/" + depth + "/" +
166         maximizing + "/" + wallsLeft1 + "/" + wallsLeft2;
167     if (cache.containsKey(key)) {
168         return cache.get(key);

```

```

165     }
166
167     Integer terminal = terminalScore(board, playerId, size,
        depth);
168     if (terminal != null) {
169         cache.put(key, terminal);
170         return terminal;
171     }
172
173     if (depth == 0) {
174         int val = evaluate(board, playerId, size,
            wallsLeft1, wallsLeft2);
175         cache.put(key, val);
176         return val;
177     }
178
179     int current = maximizing ? playerId : (3 - playerId);
180     int walls = current == 1 ? wallsLeft1 : wallsLeft2;
181     List<Move> moves = limitMoves(getMoves(board, current,
        walls));
182     consideredMoves += moves.size();
183
184     int best = maximizing ? Integer.MIN_VALUE : Integer.
        MAX_VALUE;
185     for (Move move : moves) {
186         Board copy = board.copy();
187         applyMove(copy, move);
188
189         int newWallsLeft1 = wallsLeft1, newWallsLeft2 =
            wallsLeft2;
190         if (move.getMoveType() == MoveType.PLACE_WALL) {
191             if (current == 1) {
192                 --newWallsLeft1;
193             } else {
194                 --newWallsLeft2;
195             }
196         }
197
198         int val = minimax(copy, depth - 1, !maximizing,
        playerId, size, newWallsLeft1,

```

```

        newWallsLeft2, endTime);
200         if (maximizing) {
201             best = Math.max(best, val);
202         } else {
203             best = Math.min(best, val);
204         }
205     }
206     cache.put(key, best);
207     return best;
208 }
209
210 /**
211  * ограничение ширины перебора, чтобы обычный MiniMax не за
212    висал на большом поле
213  */
214 private List<Move> limitMoves(List<Move> moves) {
215     if (moves.size() <= 16) {
216         return moves;
217     }
218     return moves.subList(0, 16);
219 }
220
221 private int scoreMoveAfterApply(Board board, Move move, int
222     playerId, int size, int wallsLeft1, int wallsLeft2) {
223     Board copy = board.copy();
224     applyMove(copy, move);
225     int newWallsLeft1 = wallsLeft1;
226     int newWallsLeft2 = wallsLeft2;
227     if (move.getMoveType() == MoveType.PLACE_WALL) {
228         if (move.getPlayerId() == 1) {
229             --newWallsLeft1;
230         } else {
231             --newWallsLeft2;
232         }
233     }
234     return evaluate(copy, playerId, size, newWallsLeft1,
235         newWallsLeft2);
236 }
237
238 private record SearchResult(Move move, int score) {

```

236 }

237 }

Реализация алгоритма AlphaBeta

Листинг Г.1 — Класс AlphaBetaAlgorithm

```

1 public class AlphaBetaAlgorithm implements Algorithm {
2     /**
3      * transposition table хранит оценку позиции вместе с глуби
4      * ной и типом границы
5      */
6     private final Map<String, TranspositionEntry>
7         transpositionTable = new HashMap<>();
8     /**
9      * ходы, которые часто приводят к хорошим отсечениям
10     */
11     private final Map<String, Integer> goodMoves = new HashMap
12         <>();
13     /**
14      * лучшие ходы, которые приводят к  $\beta \leq \alpha$ 
15     */
16     private final Move[][] bestMoves = new Move[50][2];
17     private AlgorithmReport lastReport;
18     private long nodesVisited;
19     private long consideredMoves;
20     private long cutoffs;
21     private long tableHits;
22     private List<Position> recentPositions = List.of();
23
24     @Override
25     public ComputerAlgorithmType getType() {
26         return ComputerAlgorithmType.ALPHABETA;
27     }
28
29     @Override
30     public AlgorithmReport getLastReport() {
31         return lastReport;
32     }
33
34     @Override

```

```

32     public void setRecentPositions(List<Position>
        recentPositions) {
33         this.recentPositions = recentPositions == null ? List.
            of() : List.copyOf(recentPositions);
34     }
35
36     /**
37      * пока есть время, итеративно увеличиваем глубину поиска р
        ешения
38      * если время кончилось -> отдаем лучшее, что нашли
39      * AlphaBeta ищет глубже MiniMax за счет сортировки ходов и
        отсечений
40      */
41     @Override
42     public Move getMove(Board board,
        ComputerPlayerHardnessLevel level, int playerId, int
        wallsLeft1, int wallsLeft2) {
43         long startedAt = System.currentTimeMillis();
44         int size = (int) Math.sqrt(board.getTiles().size());
45         int maxDepth = getMaxDepth(level, size);
46         long timeLimit = getTimeLimit(level, size);
47         long endTime = System.currentTimeMillis() + timeLimit;
48
49         transpositionTable.clear();
50         nodesVisited = 0;
51         consideredMoves = 0;
52         cutoffs = 0;
53         tableHits = 0;
54
55         int currentWalls = playerId == 1 ? wallsLeft1 :
            wallsLeft2;
56         Move tacticalMove = findEndgameMove(board, playerId,
            currentWalls);
57         if (tacticalMove != null) {
58             int tacticalScore = scoreMoveAfterApply(board,
                tacticalMove, playerId, size, wallsLeft1,
                wallsLeft2);
59             lastReport = new AlgorithmReport(
60                 getType().getDescription(),
61                 tacticalMove,

```

```

62         tacticalScore,
63         0,
64         nodesVisited,
65         1,
66         cutoffs,
67         tableHits,
68         System.currentTimeMillis() - startedAt,
69         "AlphaBeta_применил_эндшпильное_правило:_не
           медленная_победа_или_срочная_блокировка"
70     );
71     return tacticalMove;
72 }
73
74 Move bestMove = null;
75 int bestScore = 0;
76 int reachedDepth = 0;
77 for (int depth = 1; depth <= maxDepth; ++depth) {
78     if (System.currentTimeMillis() > endTime) {
79         break;
80     }
81     SearchResult result = search(board, depth, playerId
           , size, wallsLeft1, wallsLeft2, endTime);
82     if (result.move() != null) {
83         bestMove = result.move();
84         bestScore = result.score();
85         reachedDepth = depth;
86     }
87 }
88 lastReport = new AlgorithmReport(
89     getType().getDescription(),
90     bestMove,
91     bestScore,
92     reachedDepth,
93     nodesVisited,
94     consideredMoves,
95     cutoffs,
96     tableHits,
97     System.currentTimeMillis() - startedAt,
98     "AlphaBeta_использует_сортировку_ходов,_beta_<=
           _alpha_отсечения_и_transposition_table"

```

```

99         + "_c_depth-bound_flags;_попаданий_в_та
           блицу:_" + tableHits
100        + ";_лимит_времени:_" + timeLimit + "_м
           с"
101    );
102    return bestMove;
103 }
104
105 private int getMaxDepth(ComputerPlayerHardnessLevel level,
    int size) {
106     boolean smallBoard = size <= GameSize.SMALL.
        getAmountOfTilesPerSide();
107     boolean largeBoard = size > GameSize.NORMAL.
        getAmountOfTilesPerSide();
108     return switch (level) {
109         case EASY -> smallBoard ? 5 : 4;
110         case MEDIUM -> smallBoard ? 7 : 6;
111         case HARD -> largeBoard ? 7 : 8;
112     };
113 }
114
115 @Override
116 public long getTimeLimit(ComputerPlayerHardnessLevel level,
    int size) {
117     boolean largeBoard = size > GameSize.NORMAL.
        getAmountOfTilesPerSide();
118     boolean smallBoard = size <= GameSize.SMALL.
        getAmountOfTilesPerSide();
119     return switch (level) {
120         case EASY -> smallBoard ? 1000L : 1300L;
121         case MEDIUM -> largeBoard ? 4800L : 3600L;
122         case HARD -> largeBoard ? 9500L : 7600L;
123     };
124 }
125
126 /**
127  * генерируем всевозможные ходы, сортируем и отбираем из ни
    х топ кандидатов
128  * потом прогоняем через alpha-beta функцию оценки
129  */

```

```

130     private SearchResult search(Board board, int depth, int
        playerId, int size,
131         int wallsLeft1, int wallsLeft2,
            long endTime) {
132     List<Move> moves = new ArrayList<>(getMoves(board,
        playerId, wallsLeft1).stream()
133         .filter(move -> isRootMoveLegal(board, move,
            playerId, wallsLeft1))
134         .toList());
135     orderMoves(moves, board, playerId, size, wallsLeft1,
        wallsLeft2, 0, null);
136     if (moves.size() > 36) {
137         moves = moves.subList(0, 36);
138     }
139     consideredMoves += moves.size();
140
141     Move bestMove = null;
142     int best = Integer.MIN_VALUE;
143     for (Move move : moves) {
144         if (System.currentTimeMillis() > endTime) {
145             break;
146         }
147         Board copy = board.copy();
148         applyMove(copy, move);
149
150         int newWallsLeft1 = wallsLeft1, newWallsLeft2 =
            wallsLeft2;
151         if (move.getMoveType() == MoveType.PLACE_WALL) {
152             if (playerId == 1) {
153                 --newWallsLeft1;
154             } else {
155                 --newWallsLeft2;
156             }
157         }
158         int val = alphaBeta(copy, depth - 1, Integer.
            MIN_VALUE, Integer.MAX_VALUE, false,
159             playerId, size, newWallsLeft1,
                newWallsLeft2, endTime, 1);
160         val += rootMovePreference(move, board, playerId,
            size);

```

```

161         if (val > best) {
162             best = val;
163             bestMove = move;
164         }
165     }
166     return new SearchResult(bestMove, best);
167 }
168
169 /**
170  * если max = true (наш ход) - максимизируем оценку - резул
        ьтат в alpha (нижней границе)
171  * если max = false (ход противника) - минимизируем - резул
        ьтат в beta (верхней границе)
172  * если beta <= alpha -> можно не исследовать дальше, отсеч
        ение
173  */
174 private int alphaBeta(Board board, int depth, int alpha,
        int beta, boolean max,
175                        int playerId, int size, int
                        wallsLeft1, int wallsLeft2,
176                        long endTime, int ply) {
177     nodesVisited++;
178     if (System.currentTimeMillis() > endTime) {
179         return evaluate(board, playerId, size, wallsLeft1,
                        wallsLeft2);
180     }
181     String key = boardKey(board) + "/" + max + "/" +
        wallsLeft1 + "/" + wallsLeft2;
182     int originalAlpha = alpha;
183     int originalBeta = beta;
184
185     TranspositionEntry cached = transpositionTable.get(key)
        ;
186     if (cached != null && cached.depth >= depth) {
187         if (cached.bound == Bound.EXACT) {
188             tableHits++;
189             return cached.score;
190         }
191         if (cached.bound == Bound.LOWER) {
192             alpha = Math.max(alpha, cached.score);

```

```

193         } else if (cached.bound == Bound.UPPER) {
194             beta = Math.min(beta, cached.score);
195         }
196         if (alpha >= beta) {
197             tableHits++;
198             return cached.score;
199         }
200     }
201
202     Integer terminal = terminalScore(board, playerId, size,
203                                     depth);
204     if (terminal != null) {
205         transpositionTable.put(key, new TranspositionEntry(
206             depth, terminal, Bound.EXACT, null));
207         return terminal;
208     }
209
210     if (depth == 0) {
211         if (isNoisyPosition(board, playerId, size)) {
212             int calmScore = quiescence(board, alpha, beta,
213                                     max, playerId, size,
214                                     wallsLeft1, wallsLeft2, endTime, 2);
215             transpositionTable.put(key, new
216                 TranspositionEntry(depth, calmScore, Bound.
217                     EXACT, null));
218             return calmScore;
219         }
220         int eval = evaluate(board, playerId, size,
221                             wallsLeft1, wallsLeft2);
222         transpositionTable.put(key, new TranspositionEntry(
223             depth, eval, Bound.EXACT, null));
224         return eval;
225     }
226
227     int current = max ? playerId : 3 - playerId;
228     int walls = current == 1 ? wallsLeft1 : wallsLeft2;
229
230     List<Move> moves = getMoves(board, current, walls);
231     Move tableBestMove = cached == null ? null : cached.
232         bestMove;

```

```

225     orderMoves(moves, board, playerId, size, wallsLeft1,
                wallsLeft2, ply, tableBestMove);
226     if (moves.size() > 36) {
227         moves = moves.subList(0, 36);
228     }
229     consideredMoves += moves.size();
230
231     int best = max ? Integer.MIN_VALUE : Integer.MAX_VALUE;
232     Move bestMove = null;
233
234     for (Move move : moves) {
235         Board copy = board.copy();
236         applyMove(copy, move);
237
238         int newWallsLeft1 = wallsLeft1, newWallsLeft2 =
            wallsLeft2;
239         if (move.getMoveType() == MoveType.PLACE_WALL) {
240             if (current == 1) {
241                 --newWallsLeft1;
242             } else {
243                 --newWallsLeft2;
244             }
245         }
246
247         int val = alphaBeta(copy, depth - 1, alpha, beta, !
            max,
248             playerId, size, newWallsLeft1,
                newWallsLeft2, endTime, ply + 1);
249
250         if (max) {
251             if (val > best) {
252                 best = val;
253                 bestMove = move;
254             }
255             alpha = Math.max(alpha, val);
256         } else {
257             if (val < best) {
258                 best = val;
259                 bestMove = move;
260             }

```

```

261         beta = Math.min(beta, val);
262     }
263
264     if (beta <= alpha) {
265         cutoffs++;
266         updateBestMoves(move, ply);
267         updateGoodMoves(move);
268         break;
269     }
270 }
271
272 Bound bound = Bound.EXACT;
273 if (best <= originalAlpha) {
274     bound = Bound.UPPER;
275 } else if (best >= originalBeta) {
276     bound = Bound.LOWER;
277 }
278 transpositionTable.put(key, new TranspositionEntry(
279     depth, best, bound, bestMove));
280 return best;
281 }
282
283 private int quiescence(Board board, int alpha, int beta,
284     boolean max,
285     int playerId, int size, int
286     wallsLeft1, int wallsLeft2,
287     long endTime, int extensionDepth) {
288     nodesVisited++;
289     int standPat = evaluate(board, playerId, size,
290         wallsLeft1, wallsLeft2);
291     if (extensionDepth == 0 || System.currentTimeMillis() >
292         endTime) {
293         return standPat;
294     }
295
296     if (max) {
297         if (standPat >= beta) {
298             return beta;
299         }
300         alpha = Math.max(alpha, standPat);

```

```

296     } else {
297         if (standPat <= alpha) {
298             return alpha;
299         }
300         beta = Math.min(beta, standPat);
301     }
302
303     int current = max ? playerId : 3 - playerId;
304     int walls = current == 1 ? wallsLeft1 : wallsLeft2;
305     List<Move> moves = getQuiescenceMoves(board, current,
306         walls, playerId, size);
307     if (moves.isEmpty()) {
308         return standPat;
309     }
310     orderMoves(moves, board, playerId, size, wallsLeft1,
311         wallsLeft2, 0, null);
312
313     for (Move move : moves) {
314         if (System.currentTimeMillis() > endTime) {
315             break;
316         }
317         Board copy = board.copy();
318         applyMove(copy, move);
319
320         int newWallsLeft1 = wallsLeft1;
321         int newWallsLeft2 = wallsLeft2;
322         if (move.getMoveType() == MoveType.PLACE_WALL) {
323             if (current == 1) {
324                 --newWallsLeft1;
325             } else {
326                 --newWallsLeft2;
327             }
328         }
329
330         int value = quiescence(copy, alpha, beta, !max,
331             playerId, size,
332             newWallsLeft1, newWallsLeft2, endTime,
333             extensionDepth - 1);
334         if (max) {
335             if (value >= beta) {

```

```

332         cutoffs++;
333         return beta;
334     }
335     alpha = Math.max(alpha, value);
336 } else {
337     if (value <= alpha) {
338         cutoffs++;
339         return alpha;
340     }
341     beta = Math.min(beta, value);
342 }
343 }
344 return max ? alpha : beta;
345 }
346
347 /**
348  * сортируем ходы
349  */
350 private void orderMoves(List<Move> moves, Board board, int
    playerId, int size,
351     int wallsLeft1, int wallsLeft2, int
        ply, Move tableBestMove) {
352     moves.sort((a, b) -> Integer.compare(
353         scoreMove(b, board, playerId, size, wallsLeft1,
354             wallsLeft2, ply, tableBestMove),
355         scoreMove(a, board, playerId, size, wallsLeft1,
356             wallsLeft2, ply, tableBestMove)
357     ));
358 }
359
360 /**
361  * оцениваем ход для корректной сортировки с помощью функции
362    и evaluate из класса Algorithm
363  * будет плюсом наличие этого хода в истории хороших или лу
364    чших ходов
365  */
366 private int scoreMove(Move move, Board board, int playerId,
    int size,
367     int wallsLeft1, int wallsLeft2, int
        ply, Move tableBestMove) {

```

```

364         int score = 0;
365
366         if (move.equals(tableBestMove)) {
367             score += 20000;
368         }
369
370         if (ply < bestMoves.length) {
371             if (bestMoves[ply][0] != null && move.equals(
372                 bestMoves[ply][0])) {
373                 score += 10000;
374             }
375             if (bestMoves[ply][1] != null && move.equals(
376                 bestMoves[ply][1])) {
377                 score += 8000;
378             }
379         }
380
381         score += goodMoves.getOrDefault(move.toString(), 0) *
382             2;
383         score += rootMovePreference(move, board, playerId, size
384             );
385
386         score += scoreMoveAfterApply(board, move, playerId,
387             size, wallsLeft1, wallsLeft2);
388
389         return score;
390     }
391
392     private int scoreMoveAfterApply(Board board, Move move, int
393         playerId, int size, int wallsLeft1, int wallsLeft2) {
394         Board copy = board.copy();
395         applyMove(copy, move);
396         int newWallsLeft1 = wallsLeft1;
397         int newWallsLeft2 = wallsLeft2;
398         if (move.getMoveType() == MoveType.PLACE_WALL) {
399             if (move.getPlayerId() == 1) {
400                 --newWallsLeft1;
401             } else {
402                 --newWallsLeft2;
403             }
404         }

```

```

398     }
399     return evaluate(copy, playerId, size, newWallsLeft1,
400                     newWallsLeft2);
401 }
402 private int rootMovePreference(Move move, Board board, int
403     playerId, int size) {
404     return movementPreference(move, board, playerId, size,
405         recentPositions);
406 }
407
408 private boolean isRootMoveLegal(Board board, Move move, int
409     playerId, int wallsLeft) {
410     if (move.getPlayerId() != playerId) {
411         return false;
412     }
413     if (move.getMoveType() == MoveType.MOVE_PLAYER) {
414         return board.getAvailableMoves(getCurrentPosition(
415             board, playerId)).contains(move.getEndPosition()
416             );
417     }
418     if (wallsLeft <= 0) {
419         return false;
420     }
421     return isWallPlaceable(board, move.getStartPosition(),
422         move.getEndPosition())
423         && isValidWall(board, move.getStartPosition(),
424             move.getEndPosition());
425 }
426
427 /**
428  * сохраняем лучший ход
429  */
430 private void updateBestMoves(Move move, int ply) {
431     if (ply >= bestMoves.length || move.equals(bestMoves[
432         ply][0])) {
433         return;
434     }
435     bestMoves[ply][1] = bestMoves[ply][0];
436     bestMoves[ply][0] = move;

```

```

429     }
430
431     /**
432      * добавляем хороший ход в историю
433      */
434     private void updateGoodMoves(Move move) {
435         goodMoves.merge(move.toString(), 1, Integer::sum);
436     }
437
438     private record SearchResult(Move move, int score) {
439     }
440
441     private enum Bound {
442         EXACT,
443         LOWER,
444         UPPER
445     }
446
447     private record TranspositionEntry(int depth, int score,
448         Bound bound, Move bestMove) {
449     }

```

Реализация алгоритма Monte Carlo

Листинг Д.1 — Класс MonteCarloAlgorithm

```

1 public class MonteCarloAlgorithm implements Algorithm {
2     /**
3      * коэффициент баланса exploration/exploitation (UCT)
4      * больше -> больше случайности, меньше -> больше жадности
5      */
6     private static final double C = 1.2;
7     private static final Random random = new Random();
8     private AlgorithmReport lastReport;
9     private List<Position> recentPositions = List.of();
10    private int rootPlayerId;
11
12    @Override
13    public ComputerAlgorithmType getType() {
14        return ComputerAlgorithmType.MONTECARLO;
15    }
16
17    @Override
18    public AlgorithmReport getLastReport() {
19        return lastReport;
20    }
21
22    @Override
23    public void setRecentPositions(List<Position>
24        recentPositions) {
25        this.recentPositions = recentPositions == null ? List.
26            of() : List.copyOf(recentPositions);
27    }
28
29    /**
30     * идем по узлам дерева:
31     * добавляем детей узлу
32     * играем случайную партию
33     * обновляем статистику входов в узел и побед из него
34     */

```

```

33  @Override
34  public Move getMove(Board board,
    ComputerPlayerHardnessLevel hardnessLevel, int playerId,
    int wallsLeft1, int wallsLeft2) {
35      long startedAt = System.currentTimeMillis();
36      int size = (int) Math.sqrt(board.getTiles().size());
37      long timeLimit = getTimeLimit(hardnessLevel, size);
38      long endTime = System.currentTimeMillis() + timeLimit;
39      int rolloutDepth = getRolloutDepth(hardnessLevel, size)
        ;
40      long iterationBudget = getIterationBudget(hardnessLevel
        , size);
41
42      long iterations = 0;
43      rootPlayerId = playerId;
44      int currentWalls = playerId == 1 ? wallsLeft1 :
        wallsLeft2;
45      Move tacticalMove = findEndgameMove(board, playerId,
        currentWalls);
46      if (tacticalMove != null) {
47          lastReport = new AlgorithmReport(
48              getType().getDescription(),
49              tacticalMove,
50              scoreMove(board, tacticalMove, playerId,
                wallsLeft1, wallsLeft2),
51              0,
52              0,
53              1,
54              0,
55              0,
56              System.currentTimeMillis() - startedAt,
57              "MCTS_применил_эндшпильное_правило_до_запус
                ка_симуляций"
58          );
59      return tacticalMove;
60  }
61
62      Node root = new Node(board.copy(), null, null, playerId
        , wallsLeft1, wallsLeft2);
63      while (System.currentTimeMillis() < endTime &&

```

```

        iterations < iterationBudget) {
64         Node node = select(root);
65         Node expanded = expand(node);
66         int winner = simulate(expanded, rolloutDepth,
            playerId, size);
67         backpropagate(expanded, winner, playerId);
68         iterations++;
69         if (root.visits > 300 && bestWinRate(root) > 0.97)
            {
70             break;
71         }
72     }
73
74     Node best = root.children.stream()
75         .max(Comparator.comparingDouble(this::
            robustChildScore))
76         .orElseThrow();
77     lastReport = new AlgorithmReport(
78         getType().getDescription(),
79         best.move,
80         evaluate(best.board, playerId, size, best.
            walls1, best.walls2),
81         rolloutDepth,
82         iterations,
83         root.children.size(),
84         0,
85         0,
86         System.currentTimeMillis() - startedAt,
87         "MCTS: _выбор_по_UCT, _rollout_c_epsilon-greedy_э
            вристикой, _немедленными_победами_и_тактическ
            ими_стенами"
88         + " _лимит_времени:_" + timeLimit + " _м
            с"
89         + " _бюджет_итераций:_" +
            iterationBudget
90     );
91     return best.move;
92 }
93
94 private int getRolloutDepth(ComputerPlayerHardnessLevel

```

```

    level, int size) {
95         return switch (level) {
96             case EASY -> size * 4;
97             case MEDIUM -> size * 7;
98             case HARD -> size * 10;
99         };
100     }
101
102     private long getIterationBudget(ComputerPlayerHardnessLevel
        level, int size) {
103         boolean largeBoard = size > GameSize.NORMAL.
            getAmountOfTilesPerSide();
104         return switch (level) {
105             case EASY -> largeBoard ? 380L : 520L;
106             case MEDIUM -> largeBoard ? 1300L : 1700L;
107             case HARD -> largeBoard ? 2600L : 3400L;
108         };
109     }
110
111     @Override
112     public long getTimeLimit(ComputerPlayerHardnessLevel level,
        int size) {
113         boolean largeBoard = size > GameSize.NORMAL.
            getAmountOfTilesPerSide();
114         return switch (level) {
115             case EASY -> largeBoard ? 1200L : 950L;
116             case MEDIUM -> largeBoard ? 3800L : 2900L;
117             case HARD -> largeBoard ? 7600L : 6200L;
118         };
119     }
120
121     /**
122      * идем вниз дерева по пути наибольшего UCT
123      */
124     private Node select(Node node) {
125         while (!node.children.isEmpty() && node.untriedMoves.
            isEmpty()) {
126             node = node.children.stream().max(Comparator.
                comparing(this::uct)).orElseThrow();
127         }

```

```

128         return node;
129     }
130
131     /**
132      * создаем всевозможные ходы из текущего узла дерева
133      */
134     private Node expand(Node node) {
135         if (node.untriedMoves.isEmpty() && !node.children.
136             isEmpty()) {
137             return node;
138         }
139
140         if (node.untriedMoves.isEmpty()) {
141             node.untriedMoves.addAll(getMoves(node.board, node.
142                 player, node.getWalls()));
143             node.untriedMoves.sort((a, b) -> Integer.compare(
144                 scoreMove(node.board, b, node.player, node.
145                     walls1, node.walls2),
146                 scoreMove(node.board, a, node.player, node.
147                     walls1, node.walls2)
148             ));
149         }
150
151         if (node.untriedMoves.isEmpty()) {
152             return node;
153         }
154
155         Move move = node.untriedMoves.remove(0);
156         Board copy = node.board.copy();
157         applyMove(copy, move);
158
159         int wallsLeft1 = node.walls1;
160         int wallsLeft2 = node.walls2;
161
162         if (move.getMoveType() == MoveType.PLACE_WALL) {
163             if (node.player == 1) {
164                 --wallsLeft1;
165             } else {
166                 --wallsLeft2;
167             }
168         }

```

```

164     }
165     Node child = new Node(copy, node, move, 3 - node.player
        , wallsLeft1, wallsLeft2);
166     node.children.add(child);
167     return child;
168 }
169
170 /**
171  * играем случайную партию из узла дерева
172  */
173 private int simulate(Node node, int maxSteps, int
    rootPlayer, int size) {
174     Board board = node.board.copy();
175     int currentPlayer = node.player;
176     int wallsLeft1 = node.walls1;
177     int wallsLeft2 = node.walls2;
178
179     for (int step = 0; step < maxSteps; step++) {
180         if (isWin(board, 1, size)) {
181             return 1;
182         }
183         if (isWin(board, 2, size)) {
184             return 2;
185         }
186
187         int walls = currentPlayer == 1 ? wallsLeft1 :
            wallsLeft2;
188         List<Move> moves = getMoves(board, currentPlayer,
            walls);
189         if (moves.isEmpty()) {
190             return 3 - currentPlayer;
191         }
192
193         Move best = pickRolloutMove(board, moves,
            currentPlayer, rootPlayer, size, wallsLeft1,
            wallsLeft2);
194
195         applyMove(board, best);
196
197         if (best.getMoveType() == MoveType.PLACE_WALL) {

```

```

198         if (currentPlayer == 1) {
199             --wallsLeft1;
200         } else {
201             --wallsLeft2;
202         }
203     }
204     currentPlayer = 3 - currentPlayer;
205 }
206
207     return evaluate(board, rootPlayer, size, wallsLeft1,
208         wallsLeft2) > 0 ? rootPlayer : (3 - rootPlayer);
209 }
210
211 /**
212  * поднимаемся вверх по дереву и обновляем количество входов
213  * в и побед
214  */
215
216 private void backpropagate(Node node, int winner, int
217     playerId) {
218     while (node != null) {
219         ++node.visits;
220         if (winner == playerId) {
221             ++node.wins;
222         }
223         node = node.parent;
224     }
225 }
226
227 /**
228  * UCT – оценка узла
229  * exploitation – доля побед
230  * exploration – исследованность узла
231  * heuristic – оценка через evaluate
232  */
233
234 private double uct(Node n) {
235     if (n.visits == 0) {
236         return Double.MAX_VALUE;
237     }
238
239     double exploitation = n.wins / n.visits;

```

```

235     double exploration = C * Math.sqrt(Math.log(n.parent.
        visits) / n.visits);
236     double heuristic = normalizedEvaluate(n.board, n.parent
        .player, n.walls1, n.walls2);
237
238     return exploitation + exploration + heuristic;
239 }
240
241 private Move pickRolloutMove(Board board, List<Move> moves,
    int currentPlayer, int rootPlayer,
242     int size, int wallsLeft1, int
        wallsLeft2) {
243     for (Move move : moves) {
244         if (move.getMoveType() == MoveType.MOVE_PLAYER
245             && move.getEndPosition().row() ==
                getTargetRow(currentPlayer, size)) {
246             return move;
247         }
248     }
249
250     int walls = currentPlayer == 1 ? wallsLeft1 :
        wallsLeft2;
251     if (walls > 0 && calculateShortestPath(board,
        getCurrentPosition(board, 3 - currentPlayer),
252         getTargetRow(3 - currentPlayer, size)) <= 2) {
253         Move tacticalWall = moves.stream()
254             .filter(move -> move.getMoveType() ==
                MoveType.PLACE_WALL)
255             .max(Comparator.comparingInt(move ->
                wallImpact(board, move, currentPlayer,
                    size)))
256             .orElse(null);
257         if (tacticalWall != null && wallImpact(board,
            tacticalWall, currentPlayer, size) > 0) {
258             return tacticalWall;
259         }
260     }
261
262     if (random.nextDouble() < 0.22) {
263         return moves.get(random.nextInt(moves.size()));
    
```

```

264     }
265
266     int sampleSize = Math.min(18, moves.size());
267     List<Move> sampled = new ArrayList<>(sampleSize);
268     Set<Integer> usedIndexes = new HashSet<>();
269     while (sampled.size() < sampleSize) {
270         int index = random.nextInt(moves.size());
271         if (usedIndexes.add(index)) {
272             sampled.add(moves.get(index));
273         }
274     }
275
276     sampled.sort((a, b) -> Integer.compare(
277         scoreRolloutMove(board, b, currentPlayer,
278             rootPlayer, size, wallsLeft1, wallsLeft2),
279         scoreRolloutMove(board, a, currentPlayer,
280             rootPlayer, size, wallsLeft1, wallsLeft2)
281     ));
282     int top = Math.max(1, Math.min(4, sampled.size()));
283     return sampled.get(random.nextInt(top));
284 }
285
286 private int scoreRolloutMove(Board board, Move move, int
287     currentPlayer, int rootPlayer,
288     int size, int wallsLeft1, int
289     wallsLeft2) {
290     Board tmp = board.copy();
291     applyMove(tmp, move);
292     int newWallsLeft1 = wallsLeft1;
293     int newWallsLeft2 = wallsLeft2;
294     if (move.getMoveType() == MoveType.PLACE_WALL) {
295         if (currentPlayer == 1) {
296             --newWallsLeft1;
297         } else {
298             --newWallsLeft2;
299         }
300     }
301
302     int score = evaluate(tmp, rootPlayer, size,
303         newWallsLeft1, newWallsLeft2);

```

```

299         if (currentPlayer != rootPlayer) {
300             score = -score;
301         }
302         if (currentPlayer == rootPlayer) {
303             score += movementPreference(move, board,
304                                     currentPlayer, size, recentPositions);
305         }
306         return score;
307     }
308     private int scoreMove(Board board, Move move, int playerId,
309                          int wallsLeft1, int wallsLeft2) {
310         Board copy = board.copy();
311         applyMove(copy, move);
312         int size = (int) Math.sqrt(copy.getTiles().size());
313         int newWallsLeft1 = wallsLeft1;
314         int newWallsLeft2 = wallsLeft2;
315         if (move.getMoveType() == MoveType.PLACE_WALL) {
316             if (move.getPlayerId() == 1) {
317                 --newWallsLeft1;
318             } else {
319                 --newWallsLeft2;
320             }
321         }
322         int preference = playerId == rootPlayerId
323             ? movementPreference(move, board, playerId,
324                                 size, recentPositions)
325             : 0;
326         return evaluate(copy, playerId, size, newWallsLeft1,
327                       newWallsLeft2) + preference;
328     }
329     private double normalizedEvaluate(Board board, int playerId
330                                     , int wallsLeft1, int wallsLeft2) {
331         int size = (int) Math.sqrt(board.getTiles().size());
332         return Math.tanh(evaluate(board, playerId, size,
333                                 wallsLeft1, wallsLeft2) / 500.0) * 0.15;
334     }
335     private double robustChildScore(Node node) {

```

```

333         if (node.visits == 0) {
334             return 0;
335         }
336         return node.visits + node.wins / node.visits;
337     }
338
339     /**
340      * выбор лучшего ребенка через статистику побед
341      */
342     private double bestWinRate(Node root) {
343         return root.children.stream().mapToDouble(n -> n.visits
344             == 0 ? 0 : n.wins / n.visits).max().orElse(0);
345     }
346
347     /**
348      * проверяем, достиг ли игрок нужной строки поля
349      */
350     public boolean isWin(Board board, int playerId, int size) {
351         int row = getCurrentPosition(board, playerId).row();
352         return (playerId == 1 && row == 0) || (playerId == 2 &&
353             row == size - 1);
354     }
355
356     /**
357      * узел дерева
358      */
359     static class Node {
360         Board board;
361         Node parent;
362         List<Node> children = new ArrayList<>();
363         List<Move> untriedMoves = new ArrayList<>();
364         Move move;
365
366         int visits = 0;
367         double wins = 0;
368
369         int player;
370         int walls1, walls2;
371
372         Node(Board board, Node parent, Move move, int player,

```

```

371         int w1, int w2) {
372             this.board = board;
373             this.parent = parent;
374             this.move = move;
375             this.player = player;
376             this.walls1 = w1;
377             this.walls2 = w2;
378         }
379
380     int getWalls() {
381         return player == 1 ? walls1 : walls2;
382     }
383 }

```

Реализация обучения весов функции оценки

Листинг E.1 — Класс EvaluationWeightsStore

```

1 public final class EvaluationWeightsStore {
2     private static final Path FILE = Path.of("data", "
        evaluation-weights.properties");
3     private static final Path HISTORY_FILE = Path.of("data", "
        evaluation-weights-history.csv");
4     private static EvaluationWeights current;
5
6     private EvaluationWeightsStore() {
7     }
8
9     public static synchronized EvaluationWeights current() {
10         if (current == null) {
11             current = load();
12         }
13         return current;
14     }
15
16     public static synchronized EvaluationTrainingUpdate
        learnFromGame(List<EvaluationLearningSample> samples,
17
18
19         EvaluationWeights beforeWeights = current();
20         if (samples.isEmpty() || winnerId == 0) {
21             return new EvaluationTrainingUpdate(false, winnerId

```

```

        , samples.size(), new int[7], beforeWeights,
        beforeWeights);
22     }
23
24     int[] gradient = new int[7];
25     for (EvaluationLearningSample sample : samples) {
26         int reward = sample.playerId() == winnerId ? 1 :
            -1;
27         EvaluationFeatures delta = sample.delta();
28         gradient[0] += reward * delta.pathAdvantage();
29         gradient[1] += reward * delta.myMobility();
30         gradient[2] += reward * delta.enemyMobilityPenalty
            ();
31         gradient[3] += reward * delta.wallAdvantage();
32         gradient[4] += reward * delta.progressAdvantage();
33         gradient[5] += reward * delta.myEndgame();
34         gradient[6] += reward * delta.enemyEndgamePenalty()
            ;
35     }
36
37     if (isZero(gradient)) {
38         return new EvaluationTrainingUpdate(false, winnerId
            , samples.size(), gradient, beforeWeights,
            beforeWeights);
39     }
40
41     current = beforeWeights.adjustedByGameGradient(gradient
        , 1, samples.size());
42     save(current);
43     appendHistory(source, winnerId, samples.size(),
        gradient, beforeWeights, current);
44     return new EvaluationTrainingUpdate(true, winnerId,
        samples.size(), gradient, beforeWeights, current);
45 }
46
47 private static EvaluationWeights load() {
48     if (!Files.exists(FILE)) {
49         EvaluationWeights defaults = EvaluationWeights.
            defaults();
50         save(defaults);

```

```

51         return defaults;
52     }
53
54     Properties properties = new Properties();
55     try (Reader reader = Files.newBufferedReader(FILE,
56         StandardCharsets.UTF_8)) {
57         properties.load(reader);
58         return new EvaluationWeights(
59             readInt(properties, "pathAdvantage",
60                 EvaluationWeights.defaults().
61                 pathAdvantage()),
62             readInt(properties, "myMobility",
63                 EvaluationWeights.defaults().myMobility
64                 ()),
65             readInt(properties, "enemyMobility",
66                 EvaluationWeights.defaults().
67                 enemyMobility()),
68             readInt(properties, "wallAdvantage",
69                 EvaluationWeights.defaults().
70                 wallAdvantage()),
71             readInt(properties, "progressAdvantage",
72                 EvaluationWeights.defaults().
73                 progressAdvantage()),
74             readInt(properties, "myEndgame",
75                 EvaluationWeights.defaults().myEndgame()
76                 ),
77             readInt(properties, "enemyEndgame",
78                 EvaluationWeights.defaults().
79                 enemyEndgame()),
80             readLong(properties, "samples", 0)
81         );
82     } catch (IOException | IllegalArgumentException e) {
83         EvaluationWeights defaults = EvaluationWeights.
84             defaults();
85         save(defaults);
86         return defaults;
87     }
88 }
89
90 private static void save(EvaluationWeights weights) {

```

```

75         try {
76             Files.createDirectories(FILE.getParent());
77             Files.writeString(FILE, serialize(weights),
78                             StandardCharsets.UTF_8);
79         } catch (IOException ignored) {
80             // Если файл недоступен, алгоритмы продолжают работать с
81             // весами в памяти.
82         }
83     }
84     private static String serialize(EvaluationWeights weights)
85     {
86         return """
87             # Learned evaluation weights for Quoridor AI.
88             # Файл можно редактировать вручную; при следующ
89             ем сохранении комментарии будут сохранены.
90             #
91             # pathAdvantage - вес разницы кратчайших путей:
92             # путь соперника минус путь ИИ.
93             # Чем больше значение, тем сильнее ИИ стремится
94             # сокращать свой путь и удлинять путь соперника.
95             pathAdvantage=%d
96             #
97             # myMobility - вес количества доступных ходов ф
98             ишкой для ИИ.
99             # Увеличивает ценность позиций, где у ИИ больше
100             вариантов движения.
101             myMobility=%d
102             #
103             # enemyMobility - штраф за количество доступных
104             ходов фишкой у соперника.
105             # Чем больше значение, тем сильнее ИИ старается
106             ограничивать мобильность противника.
107             enemyMobility=%d
108             #
109             # wallAdvantage - вес преимущества по оставшимс
110             я стенам: стены ИИ минус стены соперника.
111             # Помогает не тратить стены без необходимости и
112             ценить запас стен в эндшпиле.
113             wallAdvantage=%d

```

```

103
104 _____#_progressAdvantage_-_вес_продвижения_к_целевой
    _____линии_относительно_соперника.
105 _____#_Нужен,_чтобы_алгоритм_не_зацикливался_на_равн
    _____ых_по_длине_пути_позициях.
106 _____progressAdvantage=%d
107
108 _____#_myEndgame_-_бонус_за_близость_ИИ_к_победной_л
    _____инии.
109 _____#_Работает_особенно_сильно,_когда_до_цели_остал
    _____ось_1-3_хода.
110 _____myEndgame=%d
111
112 _____#_enemyEndgame_-_штраф_за_близость_соперника_к_
    _____победной_линии.
113 _____#_Чем_больше_значение,_тем_охотнее_ИИ_срочно_бл
    _____окирует_финиш_соперника.
114 _____enemyEndgame=%d
115
116 _____#_samples_-_сколько_ходов_ИИ_участвовало_в_прин
    _____ятых_обновлениях_после_завершения_партий.
117 _____#_Весы_меняются_не_после_отдельного_хода,_а_пос
    _____ле_партии,_на_основании_победителя.
118 _____samples=%d
119 _____"".formatted(
120         weights.pathAdvantage(),
121         weights.myMobility(),
122         weights.enemyMobility(),
123         weights.wallAdvantage(),
124         weights.progressAdvantage(),
125         weights.myEndgame(),
126         weights.enemyEndgame(),
127         weights.samples()
128     );
129 }
130
131 private static void appendHistory(String source, int
    winnerId, int sampleCount, int[] gradient,
132                                     EvaluationWeights before,
    EvaluationWeights

```

```

after) {
133     try {
134         Files.createDirectories(HISTORY_FILE.getParent());
135         if (!Files.exists(HISTORY_FILE)) {
136             Files.writeString(HISTORY_FILE, historyHeader()
137                               , StandardCharsets.UTF_8);
138         }
139         Files.writeString(HISTORY_FILE, historyRow(source,
140             winnerId, sampleCount, gradient, before, after),
141             StandardCharsets.UTF_8, StandardOpenOption.
142                 APPEND);
143     } catch (IOException ignored) {
144         // История полезна для диплома, но недоступность файла не
145         // должна ломать обучение.
146     }
147 }
148
149 private static String historyHeader() {
150     return "timestamp,source,winnerId,sampleCount,"
151         + "gradientPath,gradientMyMobility,
152           gradientEnemyMobility,gradientWall,
153           gradientProgress,gradientMyEndgame,
154           gradientEnemyEndgame,"
155         + "beforePath,beforeMyMobility,
156           beforeEnemyMobility,beforeWall,
157           beforeProgress,beforeMyEndgame,
158           beforeEnemyEndgame,beforeSamples,"
159         + "afterPath,afterMyMobility,afterEnemyMobility
160           ,afterWall,afterProgress,afterMyEndgame,
161           afterEnemyEndgame,afterSamples\n";
162 }
163
164 private static String historyRow(String source, int
165     winnerId, int sampleCount, int[] gradient,
166     EvaluationWeights before,
167     EvaluationWeights after
168 ) {
169     return String.join(", ",
170         LocalDateTime.now().toString(),
171         escape(source),

```

```

157         Integer.toString(winnerId),
158         Integer.toString(sampleCount),
159         Integer.toString(gradient[0]),
160         Integer.toString(gradient[1]),
161         Integer.toString(gradient[2]),
162         Integer.toString(gradient[3]),
163         Integer.toString(gradient[4]),
164         Integer.toString(gradient[5]),
165         Integer.toString(gradient[6]),
166         Integer.toString(before.pathAdvantage()),
167         Integer.toString(before.myMobility()),
168         Integer.toString(before.enemyMobility()),
169         Integer.toString(before.wallAdvantage()),
170         Integer.toString(before.progressAdvantage()),
171         Integer.toString(before.myEndgame()),
172         Integer.toString(before.enemyEndgame()),
173         Long.toString(before.samples()),
174         Integer.toString(after.pathAdvantage()),
175         Integer.toString(after.myMobility()),
176         Integer.toString(after.enemyMobility()),
177         Integer.toString(after.wallAdvantage()),
178         Integer.toString(after.progressAdvantage()),
179         Integer.toString(after.myEndgame()),
180         Integer.toString(after.enemyEndgame()),
181         Long.toString(after.samples())
182     ) + "\n";
183 }
184
185 private static String escape(String value) {
186     String safe = value == null ? "" : value.replace("\\",
187         "\\");
188     return "\"" + safe + "\"";
189 }
190
191 private static int readInt(Properties properties, String
192     key, int fallback) {
193     return Integer.parseInt(properties.getProperty(key,
194         Integer.toString(fallback)));
195 }

```

```
194     private static long readLong(Properties properties, String
        key, long fallback) {
195         return Long.parseLong(properties.getProperty(key, Long.
            toString(fallback)));
196     }
197
198     private static boolean isZero(int[] values) {
199         for (int value : values) {
200             if (value != 0) {
201                 return false;
202             }
203         }
204         return true;
205     }
206 }
```